

La necesidad de un servicio confiable

En el nivel más bajo, las redes de comunicación de computadoras ofrecen una entrega de paquetes no confiable. Los paquetes pueden perderse cuando ocurren errores de transmisión o cuando falla el hardware de red. Pueden retrasarse cuando las redes se sobrecargan. Los sistemas de conmutación de paquetes cambian de ruta dinámicamente, lo que significa que pueden entregar los paquetes desordenados, entregarlos con un retraso considerable o entregar duplicados. En el nivel más alto, los programas de aplicación a menudo necesitan enviar grandes volúmenes de datos de una computadora a otra. Utilizar un sistema de entrega no orientado a conexión y no confiable para transferencias grandes se vuelve tedioso y engorroso; los programadores deben incorporar detección y recuperación de errores en cada aplicación. Dado que resulta difícil diseñar, comprender e implementar software que brinde confiabilidad de manera correcta, los investigadores en redes han trabajado para crear una solución de propósito general: un único protocolo de transferencia confiable que todas las aplicaciones puedan usar. Contar con un protocolo general de este tipo significa que los programadores de aplicaciones no necesitan incorporar un protocolo de transferencia confiable en cada aplicación.

Propiedades del servicio de entrega confiable

El servicio de transferencia confiable que TCP ofrece a las aplicaciones puede caracterizarse por cinco características que se describen a continuación:

1. Orientado a flujo (Stream)
2. Conexión de circuito virtual
3. Transferencia con buffer
4. Flujo no estructurado
5. Comunicación full duplex

Orientado a flujo. Cuando dos programas de aplicación usan TCP para transferir grandes volúmenes de datos, los datos se ven como un flujo de octetos. La aplicación en el host de destino recibe exactamente la misma secuencia de octetos que fue enviada por la aplicación en el host de origen.

Conexión de circuito virtual. Antes de que comience la transferencia de datos, tanto la aplicación que envía como la que recibe deben acordar establecer una conexión TCP. Una aplicación contacta a la otra para iniciar una conexión. El software TCP en los dos hosts se comunican enviando mensajes a través de la internet subyacente. Verifican que la transferencia esté autorizada y que ambos lados estén listos. Una vez resueltos todos los detalles, los módulos de protocolo informan a los programas de aplicación en cada extremo que se ha establecido una conexión y que la transferencia puede comenzar. TCP supervisa la transferencia de datos; si la comunicación falla por cualquier motivo (por ejemplo, porque el hardware de red a lo largo de la ruta falla), se informa a los programas de aplicación. Usamos el término circuito virtual para describir una conexión TCP, porque una conexión actúa como un circuito de hardware dedicado, aunque toda la comunicación se realice mediante paquetes.

Transferencia con buffer. Los programas de aplicación envían un flujo de datos a través de una conexión TCP, pasando repetidamente octetos de datos al software del protocolo. Al transferir datos, una aplicación usa piezas del tamaño que le resulte conveniente; las piezas

pueden ser tan pequeñas como un solo octeto. TCP coloca los datos en paquetes y envía los paquetes al destino. En el host receptor, TCP garantiza que los datos se coloquen en el orden original, de modo que la aplicación reciba los octetos exactamente en el mismo orden en que fueron enviados.

TCP tiene libertad para dividir el flujo en paquetes independientemente de las piezas que transfiere el programa de aplicación. Para hacer la transferencia más eficiente y minimizar el tráfico de red, las implementaciones normalmente recolectan suficientes datos de un flujo para llenar un datagrama razonablemente grande antes de transmitirlo a través de una internet. Así, incluso si el programa de aplicación genera el flujo un octeto a la vez, la transferencia a través de una internet puede ser eficiente. De manera similar, si el programa de aplicación decide generar bloques de datos extremadamente grandes, el software del protocolo puede optar por dividir cada bloque en piezas que quepan cada una en un solo paquete.

Para aplicaciones donde los datos deben transferirse sin esperar a llenar un buffer, el servicio de flujo proporciona un mecanismo de empuje (push) que las aplicaciones usan para forzar la transferencia inmediata. En el lado emisor, un push obliga al software del protocolo a transferir todos los datos que se han generado sin esperar a llenar un buffer. En el lado receptor, el push hace que TCP ponga los datos a disposición de la aplicación sin demora. La función push solo garantiza que todos los datos serán transferidos; no proporciona ningún tipo de límites (boundaries). Así, incluso cuando se fuerza la entrega, el software del protocolo puede optar por dividir el flujo de maneras inesperadas, o si una aplicación receptora es lenta, los datos de varios paquetes empujados pueden entregarse a la aplicación todos de una vez.

Flujo no estructurado. El servicio de flujo de TCP/IP no proporciona flujos de datos estructurados. Por ejemplo, no hay forma de que una aplicación de nómina (payroll) identifique el contenido del flujo como datos de nómina, ni el servicio de flujo puede marcar los límites entre registros de empleados. Los programas de aplicación que usan el servicio de flujo deben entender el contenido del flujo y acordar un formato de flujo antes de iniciar una conexión.

Comunicación full duplex. Las conexiones proporcionadas por el servicio de flujo TCP/IP permiten la transferencia concurrente en ambas direcciones. Dichas conexiones se denominan full duplex. Conceptualmente, una conexión full duplex consiste en dos flujos de datos independientes que fluyen en direcciones opuestas; desde el punto de vista de una aplicación, no hay interacción aparente entre los dos. TCP permite que una aplicación termine el flujo en una dirección mientras los datos continúan fluyendo en la otra dirección, convirtiendo la conexión en half duplex. La ventaja de una conexión full duplex es que el software de protocolo subyacente puede enviar información de control para un flujo de vuelta a la fuente en datagramas que transportan datos en la dirección opuesta. Dicho piggybacking reduce el tráfico de red.

Confiabilidad: confirmaciones y retransmisión

Dijimos que el servicio de entrega confiable de flujo garantiza la entrega de un flujo de datos enviado desde una computadora a otra sin duplicación ni pérdida de datos. Surge la pregunta: ¿cómo puede el software de protocolo proporcionar una transferencia confiable si el sistema

Optativa II - Integración de Redes (Intemetworking) – 2026
Teoría

de comunicación subyacente ofrece solo entrega de paquetes no confiables? La respuesta es complicada, pero los protocolos confiables se basan en una técnica fundamental conocida como confirmación positiva con retransmisión (PAR, positive acknowledgement with retransmission). La técnica requiere que el destinatario se comuniquen con la fuente, enviando de vuelta una confirmación (ACK) cada vez que los datos llegan con éxito. Cuando envía un paquete, el software emisor inicia un temporizador. Si llega una confirmación antes de que expire el temporizador, el emisor cancela el temporizador y se prepara para enviar más datos. Si el temporizador expira antes de que llegue una confirmación, el emisor retransmite el paquete.

Antes de poder entender el mecanismo de retransmisión de TCP, debemos considerar algunos conceptos básicos. El esquema de retransmisión más simple posible espera a que un paquete dado sea confirmado antes de enviar el siguiente paquete. Conocido como enviar-y-esperar (send-and-wait), este enfoque solo puede enviar un paquete a la vez. La figura 11.1 ilustra el intercambio de paquetes al usar enviar-y-esperar.

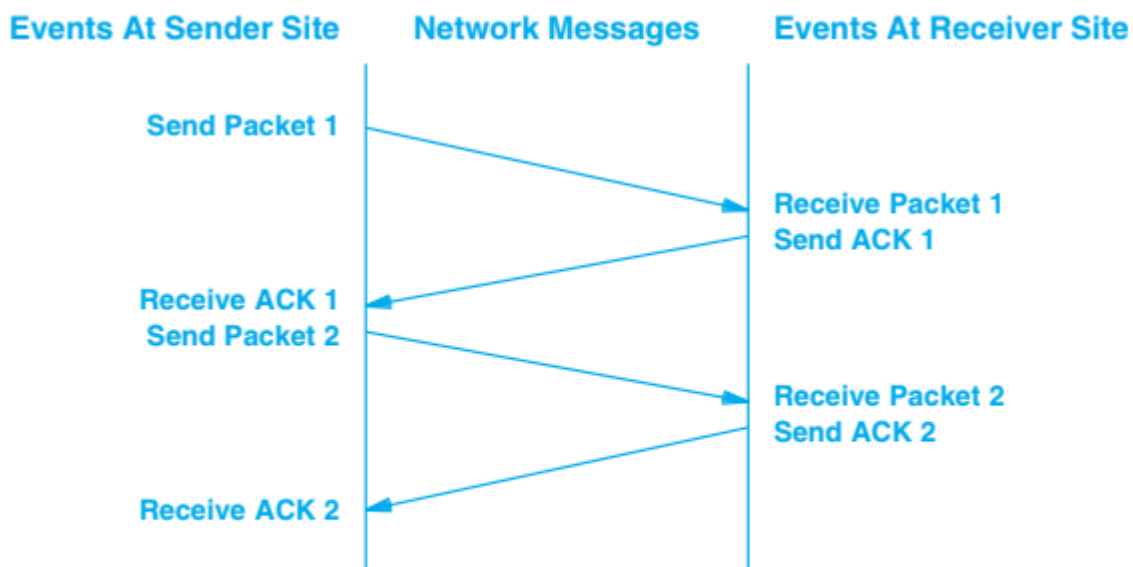


Figure 11.1 The packet exchange for a basic send-and-wait protocol. Time proceeds down the figure.

El lado izquierdo de la figura muestra los eventos en el host emisor, y el lado derecho de la figura muestra los eventos en el host receptor. Cada línea diagonal que cruza el medio representa la transferencia de un paquete o una confirmación (ACK).

La figura 11.2 usa el mismo formato que la figura 11.1 para mostrar qué sucede cuando se pierde un paquete. El emisor transmite un paquete e inicia un temporizador. El paquete se pierde, lo que significa que no llegará ninguna confirmación. Cuando el temporizador expira, el emisor retransmite el paquete perdido. El emisor debe iniciar un temporizador después de la retransmisión por si la segunda copia también se pierde. En la figura, la segunda copia llega intacta, lo que significa que el receptor envía una confirmación. Cuando la confirmación llega al lado emisor, el emisor cancela el temporizador.

En nuestra descripción anterior, un emisor debe conservar una copia de un paquete que ha sido transmitido por si el paquete debe retransmitirse. En la práctica, un emisor solo necesita

conservar los datos que van en el paquete junto con la información suficiente para permitirle reconstruir los encabezados del paquete. La idea de mantener datos sin confirmar es importante en TCP.

Aunque maneja la pérdida de paquetes o el retraso excesivo, el mecanismo de confirmación descrito anteriormente no resuelve todos los problemas: un paquete puede duplicarse. Los duplicados pueden surgir si un retraso excesivo hace que un emisor retransmita innecesariamente. Resolver la duplicación requiere pensar cuidadosamente, porque tanto los paquetes como las confirmaciones pueden duplicarse. Generalmente, los protocolos confiables detectan paquetes duplicados asignando a cada paquete un número de secuencia y exigiendo al receptor que recuerde qué números de secuencia ha recibido. Para evitar ambigüedades, los protocolos de confirmación positiva disponen que cada confirmación contenga el número de secuencia del paquete que llegó. Así, cuando llega una confirmación, esta se asocia fácilmente con un paquete en particular.

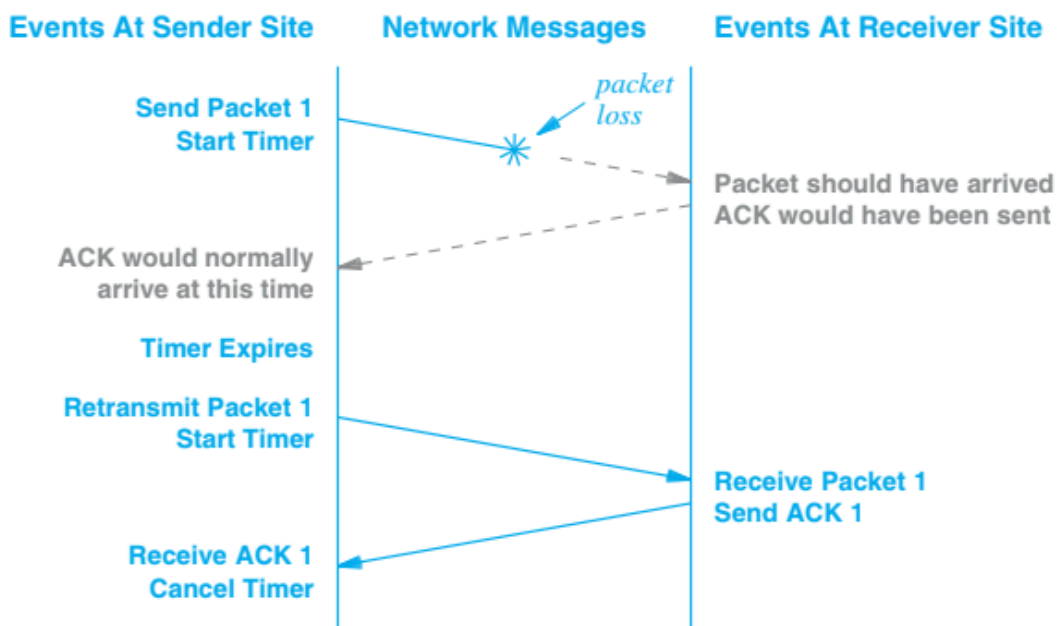


Figure 11.2 Illustration of timeout and retransmission when a packet is lost.

El paradigma de la ventana deslizante

Antes de examinar el servicio de flujo de TCP, necesitamos explorar un mecanismo adicional que sustenta la transmisión confiable. Conocido como ventana deslizante, el mecanismo mejora el rendimiento general (throughput). Para entender la motivación de las ventanas deslizantes, recordemos la secuencia de eventos de la figura 11.1. Para lograr confiabilidad, el emisor transmite un paquete y luego espera una confirmación antes de transmitir otro. Como muestra la figura, los datos fluyen entre las máquinas un paquete a la vez. La red permanecerá completamente inactiva hasta que regrese la confirmación. Si imaginamos una red con retrasos de transmisión altos, el problema se vuelve claro:

Un protocolo simple de confirmación positiva desperdicia una cantidad sustancial de capacidad de red porque debe retrasar el envío de un nuevo paquete hasta que recibe una confirmación del paquete anterior.

La técnica de ventana deslizante usa una forma más compleja de confirmación positiva con retransmisión. La idea clave es que una ventana deslizante permite a un emisor transmitir múltiples paquetes antes de esperar una confirmación. La forma más fácil de visualizar una ventana deslizante en acción es pensar en una secuencia de paquetes a transmitir como muestra la figura 11.3. El protocolo coloca una ventana pequeña de tamaño fijo sobre la secuencia y transmite todos los paquetes que se encuentran dentro de la ventana.

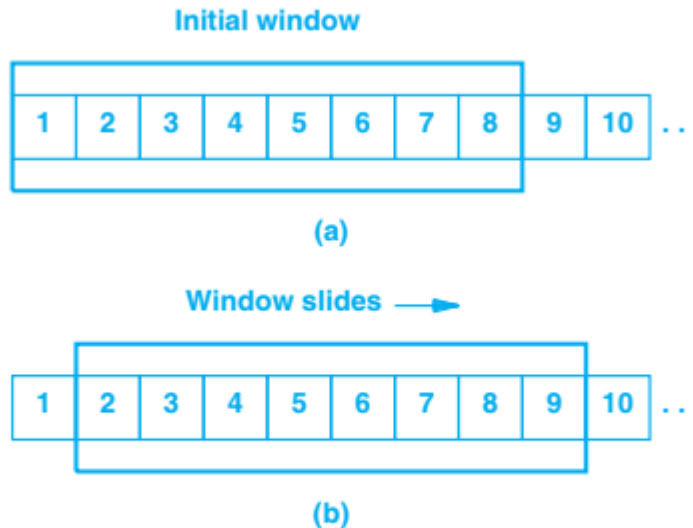


Figure 11.3 (a) A sliding window with eight packets in the window, and (b) the window sliding so that packet 9 can be sent because an acknowledgement has been received for packet 1.

Decimos que un paquete está sin confirmar (unacknowledged) si ha sido transmitido pero no se ha recibido ninguna confirmación. Técnicamente, la cantidad de paquetes que pueden estar sin confirmar en un momento dado está limitada por el tamaño de la ventana, que se restringe a un número pequeño y fijo. Por ejemplo, en un protocolo de ventana deslizante con tamaño de ventana 8, al emisor se le permite transmitir 8 paquetes antes de recibir una confirmación. Como muestra la figura 11.3, una vez que el emisor recibe una confirmación para el primer paquete dentro de la ventana, la "desliza" hacia adelante y envía el siguiente paquete. La ventana se desliza hacia adelante cada vez que llega una confirmación.

El rendimiento de los protocolos de ventana deslizante depende del tamaño de la ventana y de la velocidad a la que la red acepta paquetes. La figura 11.4 muestra un ejemplo del funcionamiento de un protocolo de ventana deslizante para un tamaño de ventana de tres paquetes. Nótese que el emisor transmite los tres paquetes antes de recibir alguna confirmación.

Con un tamaño de ventana de 1, un protocolo de ventana deslizante es exactamente igual a nuestro protocolo simple de confirmación positiva. Al aumentar el tamaño de la ventana, es posible eliminar completamente el tiempo de inactividad de la red. Es decir, en estado estable, el emisor puede transmitir paquetes tan rápido como la red pueda transferirlos. Para ver la ventaja de la ventana deslizante, comparemos la velocidad a la que se transfieren los datos en las figuras 11.1 y 11.4. El punto principal es:

Optativa II - Integración de Redes (Internetworking) – 2026
Teoría

Debido a que un protocolo de ventana deslizante bien ajustado mantiene la red completamente saturada de paquetes, puede obtener un rendimiento (throughput) sustancialmente mayor que un protocolo simple de confirmación positiva.

Conceptualmente, un protocolo de ventana deslizante siempre recuerda qué paquetes han sido confirmados y mantiene un temporizador separado para cada paquete sin confirmar. Si un paquete se pierde, el temporizador expira y el emisor retransmite ese paquete. Cuando el emisor desliza su ventana, avanza más allá de todos los paquetes confirmados. En el extremo receptor, el software de protocolo mantiene una ventana análoga, aceptando y confirmando paquetes a medida que llegan. Así, la ventana divide la secuencia de paquetes en tres conjuntos: los paquetes a la izquierda de la ventana han sido transmitidos, recibidos y confirmados exitosamente; los paquetes a la derecha aún no han sido transmitidos; y los paquetes que se encuentran en la ventana están siendo transmitidos. El paquete con el número más bajo dentro de la ventana es el primer paquete de la secuencia que no ha sido confirmado.

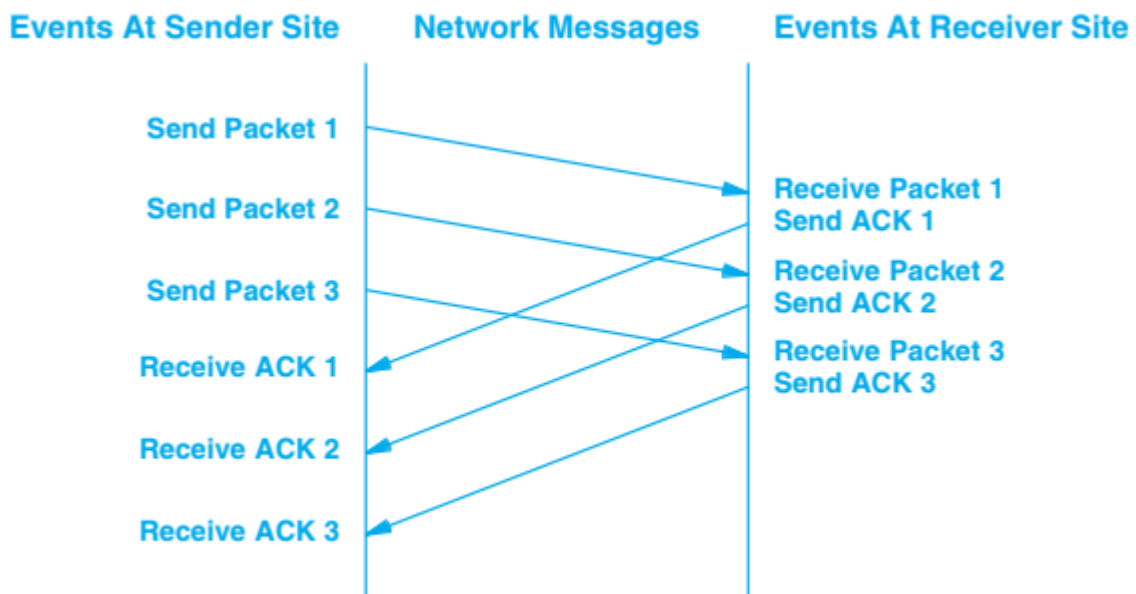


Figure 11.4 An example of sliding window with a window size of three.

El Protocolo de control de transmisión

Ahora que entendemos el principio de las ventanas deslizantes, podemos examinar el Protocolo de Control de Transmisión (TCP, Transmission Control Protocol), el protocolo que proporciona el servicio confiable de flujo. El servicio de flujo es tan significativo que a todo el conjunto de protocolos de Internet se le llama TCP/IP. Haremos una distinción clave entre el protocolo TCP y el software que implementa TCP. El protocolo TCP proporciona una especificación análoga a un plano (blueprint); el software TCP implementa la especificación. Aunque a veces resulta conveniente pensar en TCP como una pieza de software, los lectores deben reconocer la distinción: TCP es un protocolo de comunicación, no una pieza de software.

¿Qué proporciona exactamente TCP? TCP es complejo, así que no hay una respuesta simple. El protocolo especifica el formato de los datos y las confirmaciones que dos computadoras intercambian para lograr una transferencia confiable, así como los procedimientos que las computadoras usan para asegurar que los datos lleguen correctamente. Especifica cómo el software TCP distingue entre múltiples destinos en una máquina dada, y cómo las máquinas que se comunican se recuperan de errores como paquetes perdidos o duplicados. El protocolo también especifica cómo dos computadoras inician una conexión TCP y cómo acuerdan cuándo esta ha finalizado.

También es importante entender qué es lo que el protocolo no incluye. Aunque la especificación de TCP describe en términos generales cómo los programas de aplicación usan TCP, no dicta los detalles de la interfaz entre un programa de aplicación y TCP. Es decir, la documentación del protocolo solo trata las operaciones que TCP proporciona; no especifica los procedimientos exactos que las aplicaciones invocan para acceder a esas operaciones. La razón para dejar sin especificar la interfaz de aplicación es la flexibilidad. En particular, dado que el software TCP forma parte del sistema operativo de una computadora, necesita emplear cualquier interfaz que el sistema operativo proporcione. Permitir flexibilidad a los implementadores hace posible tener una única especificación de TCP que pueda usarse para construir software para una variedad de sistemas de cómputo.

Debido a que no hace suposiciones sobre el sistema de comunicación subyacente, TCP puede usarse a través de una amplia variedad de redes subyacentes. Puede funcionar a través de un satélite con largos retrasos, una red inalámbrica donde la interferencia causa la pérdida de muchos paquetes, o una conexión arrendada en la que los retrasos varían dramáticamente según la congestión actual. Su capacidad para adaptarse a una gran variedad de redes subyacentes es una de las fortalezas de TCP.

Capas, puertos, conexiones y puntos finales

TCP, que reside en la capa de transporte justo por encima de IP, permite que múltiples programas de aplicación en una computadora dada se comuniquen de manera concurrente, y demultiplexa el tráfico TCP entrante entre las aplicaciones. Así, en términos del modelo de capas, TCP es un par conceptual de UDP, como muestra la figura 11.5:

Conceptual Layering

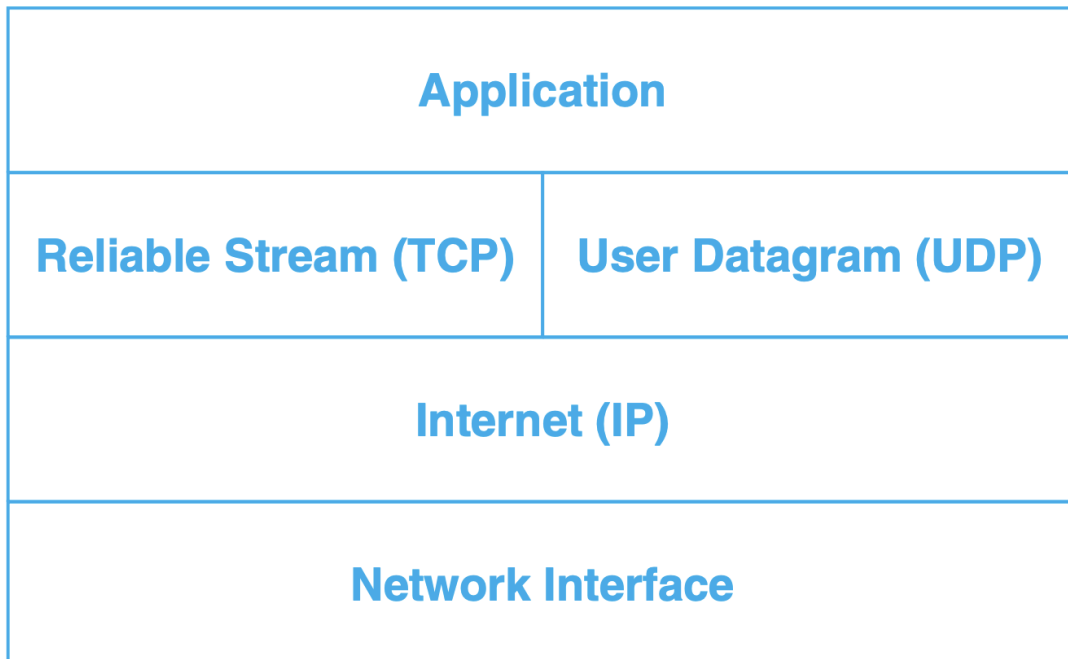


Figure 11.5 The conceptual layering of UDP and TCP above IP.

Aunque están en la misma capa conceptual, TCP y UDP proporcionan servicios completamente diferentes. Al igual que el Protocolo de Datagramas de Usuario (UDP), TCP usa números de puerto de protocolo para identificar programas de aplicación. También como UDP, un número de puerto TCP tiene una longitud de dieciséis bits. A cada puerto TCP se le asigna un pequeño entero usado para identificarlo. Es importante entender que los puertos TCP son conceptualmente independientes de los puertos UDP, una aplicación puede usar el puerto UDP 30.000 mientras que otra aplicación usa el puerto TCP 30.000. Dijimos que un puerto UDP consiste en una cola que contiene los datagramas entrantes. Los puertos TCP son mucho más complejos porque un solo número de puertos no identifica una aplicación. En cambio, TCP ha sido diseñado sobre una abstracción de conexión en la que los objetos a identificar son las conexiones TCP, no los puertos individuales. Cada conexión TCP se especifica mediante un par de puntos finales (endpoints) que corresponden al par de aplicaciones que se comunican. Entender que TCP usa la noción de conexiones es crucial porque ayuda a explicar el significado y el uso de los números de puerto TCP: TCP usa la conexión, no el puerto de protocolo, como su abstracción fundamental; las conexiones se identifican mediante un par de puntos finales.

¿Qué son exactamente los "puntos finales" de una conexión TCP? Hemos dicho que una conexión consiste en un circuito virtual entre dos programas de aplicación, por lo que podría parecer natural suponer que un programa de aplicación sirve como el punto final de la conexión. No es así. En cambio, TCP define un punto final como un par de enteros (host, puerto), donde host es la dirección IP de un host y puerto es un puerto TCP en ese host. Por ejemplo, un punto final IPv4 (128.10.2.3, 25) especifica el puerto TCP 25 en la máquina con dirección IPv4 128.10.2.3.

Ahora que hemos definido los puntos finales, es fácil entender las conexiones TCP. Recordemos que una conexión se define por sus dos puntos finales. Así, si hay una conexión desde la máquina (18.26.0.36) en el MIT hacia la máquina (128.10.2.3) en la Universidad de Purdue, podría definirse mediante los puntos finales: (18.26.0.36, 1069) y (128.10.2.3, 25). Mientras tanto, otra conexión podría estar en curso desde la máquina (128.9.0.32) en el Information Sciences Institute hacia la misma máquina en Purdue, identificada por sus puntos finales: (128.9.0.32, 1184) y (128.10.2.3, 53). Hasta ahora, nuestros ejemplos de conexiones han sido sencillos porque los puertos usados en todos los puntos finales han sido únicos. Sin embargo, la abstracción de conexión permite que múltiples conexiones compartan un punto final. Por ejemplo, podríamos agregar otra conexión a las dos listadas anteriormente desde la máquina (128.2.254.139) en CMU hacia la máquina en Purdue usando los puntos finales: (128.2.254.139, 1184) y (128.10.2.3, 53).

Podría parecer extraño que dos conexiones puedan usar el puerto TCP 53 en la máquina 128.10.2.3 simultáneamente, pero no hay ambigüedad. Debido a que TCP asocia los mensajes entrantes con una conexión en lugar de con un puerto de protocolo, usa ambos puntos finales para identificar la conexión apropiada. La idea importante para recordar es: Debido a que TCP identifica una conexión mediante un par de puntos finales, un número de puerto TCP dado puede ser compartido por múltiples conexiones en la misma máquina.

Desde el punto de vista de un programador, la abstracción de conexión es significativa. Significa que un programador puede diseñar un programa que proporcione servicio concurrente a múltiples conexiones simultáneamente, sin necesitar números de puerto local únicos para cada conexión. Por ejemplo, la mayoría de los sistemas proporcionan acceso concurrente a su servicio de correo electrónico, permitiendo que múltiples computadoras les envíen correo electrónico simultáneamente. Debido a que usa TCP para comunicarse, la aplicación que acepta correo entrante solo necesita usar un puerto TCP local, aunque múltiples conexiones puedan proceder de manera concurrente.

Aperturas pasivas y activas

A diferencia de UDP, TCP es un protocolo orientado a conexión que requiere que ambos puntos finales acuerden participar. Es decir, antes de que el tráfico TCP pueda pasar a través de una internet, los programas de aplicación en ambos extremos de la conexión deben acordar que la conexión es deseada. Para hacerlo, el programa de aplicación en un extremo realiza una apertura pasiva (passive open) contactando al sistema operativo local e indicando que aceptará una conexión entrante para un número de puerto específico. El software de protocolo se prepara para aceptar una conexión en ese puerto. El programa de aplicación en el otro extremo puede entonces realizar una apertura activa (active open) solicitando que se establezca una conexión TCP. Los dos módulos de software TCP se comunican para establecer y verificar una conexión. Una vez que se ha creado una conexión, los programas de aplicación pueden comenzar a pasar datos; los módulos de software TCP en cada extremo intercambian mensajes que garantizan la entrega confiable. Volveremos a los detalles del establecimiento de conexiones después de examinar el formato de mensajes de TCP.

Segmentos, flujos y números de secuencia

TCP considera el flujo de datos como una secuencia de octetos que se divide en segmentos para su transmisión. Normalmente, cada segmento viaja a través de la internet subyacente en un único datagrama IP. TCP usa un mecanismo especializado de ventana deslizante que optimiza el rendimiento (throughput) y maneja el control de flujo. Al igual que el protocolo de ventana deslizante descrito anteriormente, el mecanismo de ventana de TCP hace posible enviar múltiples segmentos antes de que llegue una confirmación. Hacerlo aumenta el rendimiento total porque mantiene la red ocupada. Veremos que la forma TCP de un protocolo de ventana deslizante también resuelve el problema de control de flujo extremo a extremo al permitir que el receptor restrinja la transmisión hasta que tenga suficiente espacio de buffer para acomodar más datos.

El mecanismo de ventana deslizante de TCP opera a nivel de octeto, no a nivel de segmento o paquete. Los octetos del flujo de datos se numeran secuencialmente, y un emisor mantiene tres punteros asociados con cada conexión. Los punteros definen una ventana deslizante como ilustra la figura 11.6. El primer puntero marca el lado izquierdo de la ventana deslizante, separando los octetos que han sido enviados y confirmados de los octetos aún por confirmar. Un segundo puntero marca el lado derecho de la ventana deslizante y define el octeto más alto en la secuencia que puede enviarse antes de que se reciban más confirmaciones. El tercer puntero marca el límite dentro de la ventana que separa los octetos

que ya han sido enviados de aquellos que no han sido enviados. El software de protocolo envía todos los octetos en la ventana sin demora, por lo que el límite dentro de la ventana usualmente se mueve de izquierda a derecha rápidamente.

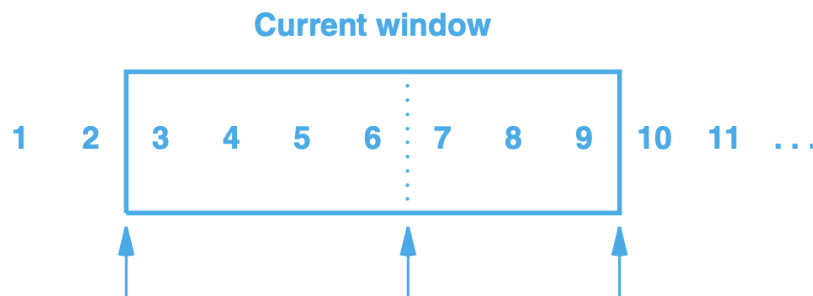


Figure 11.6 An example of TCP's sliding window where octets through 2 have been sent and acknowledged, octets 3 through 6 have been sent but not acknowledged, octets 7 through 9 have not been sent but will be sent without delay, and octets 10 and higher cannot be sent until the window moves.

Hemos descrito cómo la ventana TCP del emisor se desliza a lo largo del flujo, y mencionamos que el receptor debe mantener una ventana similar para recrear el flujo. Sin embargo, es importante entender que, dado que las conexiones TCP son full duplex, dos transferencias proceden simultáneamente sobre cada conexión, una en cada dirección. Pensamos en las transferencias como completamente independientes porque en cualquier momento los datos pueden fluir a través de la conexión en una dirección, o en ambas direcciones. Así, el software TCP en una computadora mantiene dos ventanas por conexión: una ventana se desliza a medida que se envía el flujo de datos, mientras que la otra se desliza a medida que se reciben los datos.

Tamaño de ventana variable y control de flujo

Una diferencia entre el protocolo de ventana deslizante de TCP y el protocolo de ventana deslizante simplificado presentado en la figura 11.4 surge porque TCP permite que el tamaño de la ventana varíe con el tiempo. Cada confirmación, que especifica cuántos octetos se han recibido, contiene un anuncio de ventana (window advertisement) que especifica cuántos octetos adicionales de datos está dispuesto a aceptar el receptor más allá de los datos que se están confirmando. Pensamos en el anuncio de ventana como una especificación del tamaño de buffer actual del receptor. En respuesta a un anuncio de ventana aumentado, el emisor incrementa el tamaño de su ventana deslizante y procede a enviar los octetos que aún no han sido confirmados. En respuesta a un anuncio de ventana disminuido, el emisor reduce el

tamaño de su ventana y deja de enviar octetos más allá del límite. El software TCP no debe contradecir anuncios previos reduciendo la ventana más allá de posiciones previamente aceptables en el flujo de octetos. En cambio, los anuncios más pequeños acompañan a las confirmaciones, de modo que el tamaño de la ventana solo cambia en el momento en que se desliza hacia adelante.

La ventaja de usar una ventana de tamaño variable es que proporciona la capacidad de manejar el control de flujo. Para evitar recibir más datos de los que puede almacenar, el receptor envía anuncios de ventana más pequeños a medida que su buffer se llena. En el caso extremo, el receptor anuncia un tamaño de ventana de cero para detener todas las transmisiones. Más tarde, cuando el espacio de buffer queda disponible, el receptor anuncia un tamaño de ventana distinto de cero para activar nuevamente el flujo de datos.

Contar con un mecanismo de control de flujo es esencial en un entorno donde computadoras de diversas velocidades y tamaños se comunican a través de redes y enrutadores de diversas velocidades y capacidades. Hay dos problemas independientes. Primero, los protocolos necesitan proporcionar control de flujo extremo a extremo entre la fuente y el destino final. Por ejemplo, cuando un smartphone se comunica con una supercomputadora potente, el smartphone necesita regular la afluencia de datos, o el software de protocolo se verá desbordado rápidamente. Así, TCP debe implementar control de flujo extremo a extremo para garantizar una entrega confiable. Segundo, se necesita un mecanismo que permita a los sistemas intermedios (es decir, los enrutadores) controlar a una fuente que envía más tráfico del que la máquina puede tolerar.

Cuando las máquinas intermedias se sobrecargan, la condición se llama congestión, y los mecanismos para resolver el problema se llaman mecanismos de control de congestión. TCP usa su esquema de ventana deslizante para resolver el problema de control de flujo extremo a extremo. Discutiremos el control de congestión más adelante, pero debe notarse que un protocolo bien diseñado puede detectar y recuperarse de la congestión, mientras que un protocolo mal diseñado empeorará la congestión. En particular, un esquema de retransmisión cuidadosamente elegido puede ayudar a evitar la congestión, pero un esquema mal elegido puede complicar al retransmitir agresivamente.

Formato del segmento TCP

La unidad de transferencia entre el software TCP de dos máquinas se denomina segmento. Los segmentos se intercambian para establecer una conexión, transferir datos, enviar confirmaciones, anunciar tamaños de ventana y cerrar conexiones. Debido a que TCP permite

Optativa II - Integración de Redes (Intemetworking) – 2026
Teoría

el piggybacking, una confirmación que viaja de la computadora A a la computadora B puede viajar en el mismo segmento que datos que van de la computadora A a la computadora B, aunque la confirmación se refiera a datos enviados de B a A.

Como la mayoría de los protocolos, un mensaje se divide en dos partes conceptuales: un encabezado que contiene metadatos y un área de carga útil (payload) que transporta los datos. La figura 11.7 muestra el formato del segmento TCP.

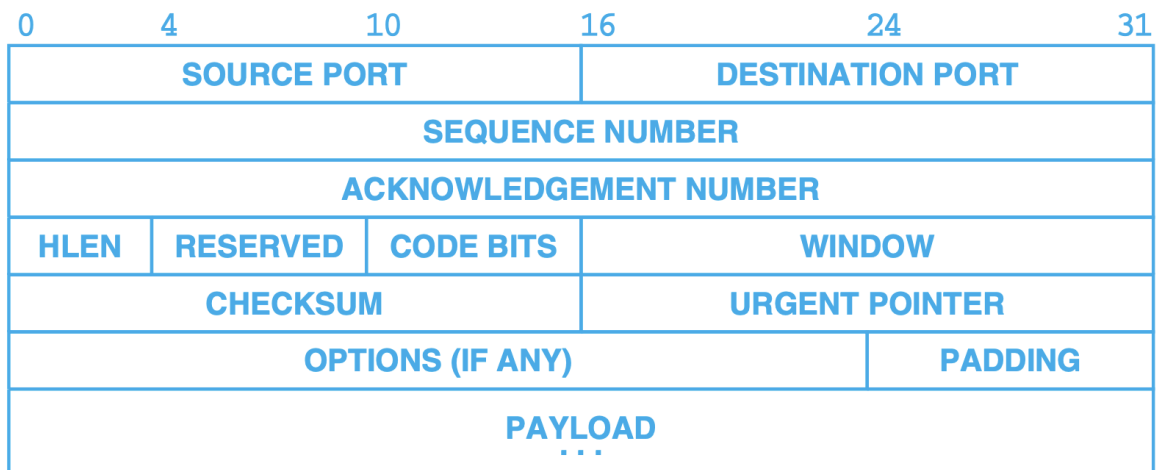


Figure 11.7 TCP segment format with a TCP header followed by a payload.

El encabezado, conocido como encabezado TCP, consta de al menos 20 octetos y puede contener más si el segmento transporta opciones. El encabezado tiene la información de identificación y control esperada. Los campos SOURCE PORT y DESTINATION PORT contienen los números de puerto TCP que identifican a los programas de aplicación en los extremos de la conexión. El campo SEQUENCE NUMBER identifica la posición, dentro del flujo de octetos del emisor, de los datos que transporta el segmento. El campo ACKNOWLEDGEMENT NUMBER identifica el número del octeto que la fuente espera recibir a continuación. Nótese que el número de secuencia se refiere al flujo que viaja en la misma dirección que el segmento, mientras que el número de confirmación se refiere al flujo que viaja en la dirección opuesta al segmento.

El campo HLEN contiene un entero que especifica la longitud del encabezado del segmento, medida en múltiplos de 32 bits. Es necesario porque el campo OPTIONS varía en longitud, dependiendo de qué opciones se incluyen. Así, el tamaño del encabezado TCP varía según las opciones seleccionadas. El campo de 6 bits marcado como RESERVED está reservado para uso futuro (una sección posterior describe un uso propuesto).

Algunos segmentos transportan solo una confirmación, mientras que otros transportan datos. Otros transportan solicitudes para establecer o cerrar una conexión. El software TCP usa el

campo de 6 bits llamado CODE BITS para determinar el propósito y el contenido del segmento. Los seis bits indican cómo interpretar otros campos del encabezado según la tabla de la figura 11.8.

El software TCP anuncia cuántos datos está dispuesto a aceptar cada vez que envía un segmento, especificando el tamaño de su buffer en el campo WINDOW. El campo contiene un entero sin signo de 16 bits en orden de bytes estándar de red (network-standard byte order). Los anuncios de ventana proporcionan un ejemplo de piggybacking, ya que acompañan a todos los segmentos, incluidos tanto los que transportan datos como los que transportan solo una confirmación.

Bit (left to right)	Meaning if bit set to 1
URG	Urgent pointer field is valid
ACK	Acknowledgement field is valid
PSH	This segment requests a push
RST	Reset the connection
SYN	Synchronize sequence numbers
FIN	Sender has reached end of its byte stream

Figure 11.8 Bits of the CODE BITS field in the TCP header.

Datos fuera de banda

Aunque TCP es un protocolo orientado a flujo, a veces es importante que el programa en un extremo de una conexión envíe datos fuera de banda (out of band), sin esperar a que el programa en el otro extremo de la conexión consuma los octetos ya presentes en el flujo. Por ejemplo, cuando TCP se usa para una aplicación de escritorio remoto, el usuario puede decidir enviar una secuencia de teclado que interrumpe o cancela el programa actualmente en ejecución. Este tipo de señales son necesarias con mayor frecuencia cuando una aplicación en la máquina remota se congela y no responde a los clics del mouse ni a las pulsaciones normales del teclado. La señal de interrupción debe enviarse sin esperar a que el programa remoto lea los octetos ya presentes en el flujo TCP (de lo contrario, no sería posible cancelar programas que dejan de leer la entrada).

Para dar cabida a la señalización fuera de banda, TCP permite que el emisor especifique datos como urgentes (urgent), lo que significa que la aplicación receptora debe ser notificada de su llegada lo más rápido posible, sin importar su posición en el flujo. El protocolo especifica

que, cuando se encuentran datos urgentes, el TCP receptor debe notificar al programa de aplicación asociado con la conexión para que entre en "modo urgente". Después de que se han consumido todos los datos urgentes, TCP le indica al programa de aplicación que vuelva a la operación normal. Los detalles exactos de cómo TCP informa a una aplicación sobre datos urgentes dependen del sistema operativo de la computadora.

El mecanismo usado para marcar datos urgentes al transmitirlos en un segmento consiste en el bit de código URG y el campo URGENT POINTER en el encabezado del segmento. Cuando el bit URG está activado, el campo URGENT POINTER especifica la posición en el segmento donde terminan los datos urgentes.

Opciones de TCP

Como indica la figura 11.7, un encabezado TCP puede contener cero o más opciones; las siguientes secciones explican las opciones disponibles. Cada opción comienza con un campo de 1 octeto que especifica el tipo de opción, seguido de un campo de longitud de 1 octeto que especifica el tamaño de la opción en octetos. Recordemos que la longitud del encabezado se especifica en múltiplos de 32 bits. Si las opciones no ocupan un múltiplo exacto de 32 bits, se agrega PADDING (relleno) al final del encabezado.

1 Opción de tamaño máximo de segmento

Un emisor puede elegir la cantidad de datos que se coloca en cada segmento. Sin embargo, ambos extremos de una conexión TCP necesitan acordar un segmento máximo que transferirán. TCP usa una opción de tamaño máximo de segmento (MSS, maximum segment size) para permitir que un receptor especifique el tamaño máximo de segmento que está dispuesto a recibir. Un sistema embebido que solo tiene unos pocos cientos de bytes de espacio de buffer puede especificar un MSS que restrinja los segmentos para que quepan en el buffer. La negociación del MSS es especialmente significativa porque permite que sistemas heterogéneos se comuniquen, una supercomputadora puede comunicarse con un pequeño nodo sensor inalámbrico. Para maximizar el rendimiento (throughput), cuando dos computadoras se conectan a la misma red física, TCP normalmente calcula un tamaño máximo de segmento tal que los datagramas IP resultantes coincidan con el MTU de la red. Si los puntos finales no se encuentran en la misma red física, pueden intentar descubrir el MTU mínimo a lo largo de la ruta entre ellos, o elegir un tamaño máximo de segmento igual al tamaño mínimo de carga útil (payload) de datagrama.

En un entorno general de internet, elegir un buen tamaño máximo de segmento puede ser difícil, porque el rendimiento puede ser deficiente tanto con tamaños de segmento

extremadamente grandes como extremadamente pequeños. Por un lado, cuando el tamaño de segmento es pequeño, la utilización de la red permanece baja. Para ver por qué, recordemos que los segmentos TCP viajan encapsulados en datagramas IP que a su vez están encapsulados en tramas de red física. Así, cada trama transporta al menos 20 octetos de encabezado TCP más 20 octetos de encabezado IP (IPv6 es más grande). Por lo tanto, los datagramas que transportan solo un octeto de datos usan como máximo $1/41$ del ancho de banda subyacente de la red para los datos que se transfieren (menos para IPv6). Por otro lado, tamaños de segmento extremadamente grandes también pueden producir un rendimiento deficiente. Los segmentos grandes resultan en datagramas IP grandes. Cuando dichos datagramas viajan a través de una ruta con un MTU pequeño, IP debe fragmentarse. A diferencia de un segmento TCP, un fragmento no puede confirmarse ni retransmitirse de manera independiente; todos los fragmentos deben llegar, o de lo contrario debe retransmitirse el datagrama completo. Si la probabilidad de perder un fragmento dado es distinta de cero, aumentar el tamaño del segmento por encima del umbral de fragmentación disminuye la probabilidad de que el datagrama llegue, lo que disminuye el rendimiento.

En teoría, el tamaño óptimo de segmento, S , ocurre cuando los datagramas IP que transportan los segmentos son tan grandes como sea posible sin requerir fragmentación en ningún punto a lo largo de la ruta, desde el origen hasta el destino. En la práctica, encontrar S significa encontrar el MTU de la ruta, lo cual implica sondeo (probing). Para una conexión TCP de corta duración (por ejemplo, donde solo se intercambian unos pocos paquetes), el sondeo puede introducir retraso. En segundo lugar, debido a que los enrutadores en una internet pueden cambiar rutas dinámicamente, la ruta que siguen los datagramas entre un par de computadoras que se comunican puede cambiar dinámicamente, y también puede cambiar el tamaño al que los datagramas deben fragmentarse. En tercer lugar, el tamaño óptimo depende de los encabezados de protocolos de nivel inferior (por ejemplo, el tamaño del segmento TCP será menor si el datagrama IP incluye opciones IPv4 o encabezados de extensión IPv6).

2 Opción de escalado de ventana

Debido a que el campo WINDOW en el encabezado TCP tiene una longitud de 16 bits, el tamaño máximo de ventana es de 64 Kbytes. Aunque la ventana fue suficiente para las primeras redes, se necesita un tamaño de ventana mayor para obtener un alto rendimiento en una red, como un canal satelital, que tiene un producto retraso-ancho de banda grande (informalmente llamado long fat pipe). Para acomodar tamaños de ventana mayores, se creó una opción de escalado de ventana para TCP. La opción consta de tres octetos: un tipo, una

longitud y un valor de desplazamiento (shift), S. En esencia, el valor de desplazamiento especifica un factor de escala binario que se aplica al valor de la ventana. Cuando el escalado de ventana está en efecto, un receptor extrae el valor del campo WINDOW, W, y desplaza W hacia la izquierda S bits para obtener el tamaño de ventana real. Varios detalles complican el diseño. La opción puede negociarse cuando se establece inicialmente la conexión, en cuyo caso se asume que todos los anuncios de ventana sucesivos usan la escala negociada, o la opción puede especificarse en cada segmento, en cuyo caso el factor de escala puede variar de un segmento a otro. Además, si alguno de los dos lados de una conexión implementa el escalado de ventana pero no necesita escalar su ventana, ese lado envía la opción con valor cero, lo que hace que el factor de escala sea 1.

3 Opción de marca de tiempo (Timestamp)

La opción de marca de tiempo de TCP fue inventada para ayudar a TCP a calcular el retraso en la red subyacente. También puede manejar el caso en que los números de secuencia de TCP excedan 2^{32} (conocido como Protección contra números de secuencia envueltos, PAWS, Protect Against Wrapped Sequence numbers). Además de los campos requeridos de tipo y longitud, una opción de marca de tiempo incluye dos valores: un valor de marca de tiempo y un valor de marca de tiempo de respuesta de eco (echo reply). Un emisor coloca la hora de su reloj actual en el campo de marca de tiempo al enviar un paquete; un receptor copia el campo de marca de tiempo en el campo de respuesta de eco antes de devolver una confirmación para el paquete. Así, cuando llega una confirmación, el emisor puede calcular con precisión el tiempo total transcurrido desde que se envió el segmento.

Cálculo del checksum de TCP

El campo CHECKSUM en el encabezado TCP es un checksum de complemento a uno de 16 bits usado para verificar la integridad tanto de los datos como del encabezado TCP. Al igual que con otros checksums, TCP usa aritmética de 16 bits y toma el complemento a uno de la suma en complemento a uno. Para calcular el checksum, el software TCP en la máquina emisora sigue un procedimiento similar al descrito en el Capítulo 10 para UDP. Conceptualmente, TCP antepone un pseudo-encabezado (pseudo-header) al segmento TCP, agrega los bits en cero necesarios para que el segmento sea un múltiplo de 16 bits, y calcula el checksum de 16 bits sobre todo el resultado. Al igual que con UDP, el pseudo-encabezado no forma parte del segmento y nunca se transmite en un paquete. En el sitio receptor, el software TCP extrae los campos del encabezado IP, reconstruye un pseudo-encabezado y realiza el mismo cálculo de checksum para verificar que el segmento llegó intacto.

El propósito de usar un pseudo-encabezado TCP es exactamente el mismo que en UDP. Permite al receptor verificar que el segmento ha llegado al punto final correcto, el cual incluye tanto una dirección IP como un número de puerto de protocolo. Tanto la dirección IP de origen como la de destino son importantes para TCP, ya que debe usarlas para identificar la conexión a la que pertenece el segmento. Por lo tanto, cada vez que llega un datagrama que transporta un segmento TCP, IP debe pasarle a TCP las direcciones IP de origen y destino del datagrama, así como el segmento en sí. La figura 11.9 muestra el formato del pseudo-encabezado usado en el cálculo del checksum para IPv4, y la figura 11.10 muestra el formato para un pseudo-encabezado IPv6.

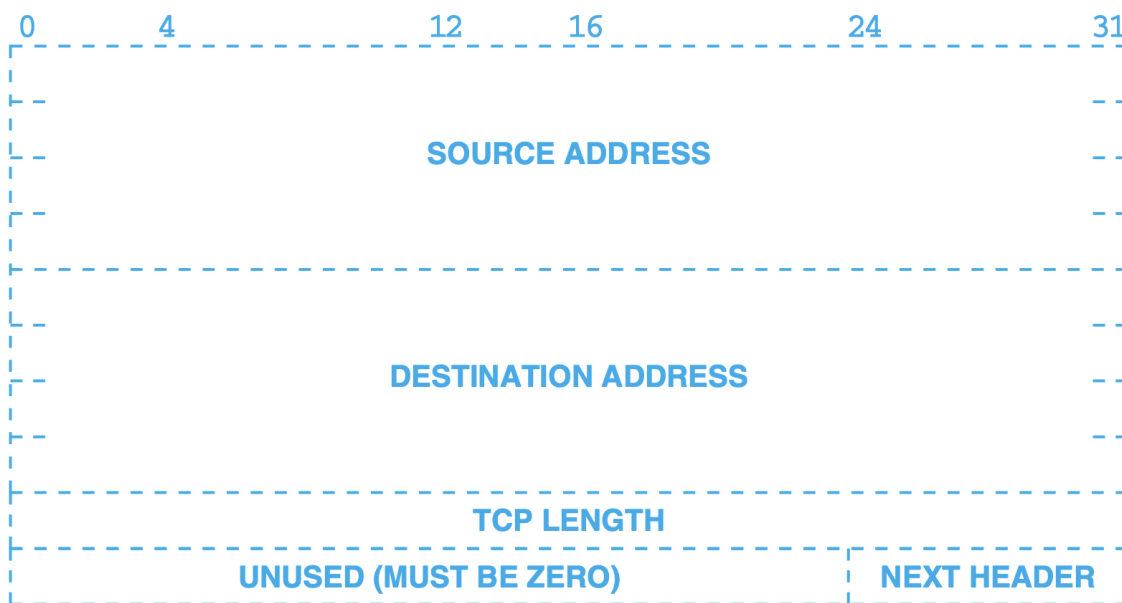


Figure 11.10 The 40 octets of the IPv6 pseudo-header used in TCP checksum computations.

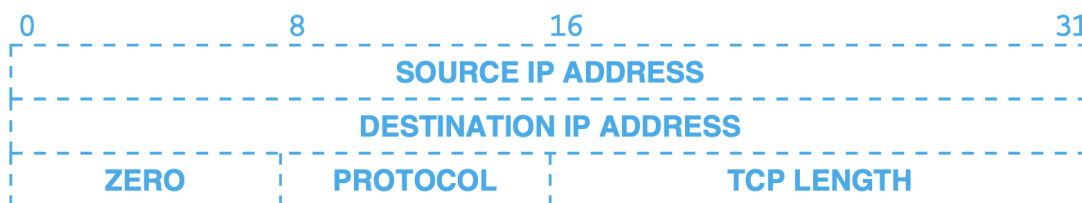


Figure 11.9 The 12 octets of the IPv4 pseudo-header used in TCP checksum computations.

Por supuesto, el pseudo-encabezado IPv4 usa direcciones IPv4 de origen y destino, y el pseudo-encabezado IPv6 usa direcciones IPv6. Al campo PROTOCOL (IPv4) o al campo NEXT HEADER (IPv6) se le asigna el valor 6, el valor correspondiente a los datagramas que

transportan TCP. El campo TCP LENGTH especifica la longitud total del segmento TCP, incluyendo el encabezado TCP.

Confirmaciones, retransmisión y tiempos de espera (timeouts)

Debido a que TCP envía datos en segmentos de longitud variable, y debido a que los segmentos retransmitidos pueden incluir más datos que el original, una confirmación no puede referirse fácilmente a un datagrama o a un segmento. En cambio, una confirmación se refiere a una posición en el flujo usando los números de secuencia del flujo. El receptor recolecta octetos de datos de los segmentos que llegan y reconstruye una copia exacta del flujo que se está enviando. Debido a que los segmentos viajan en datagramas IP, pueden perderse o entregarse fuera de orden; el receptor usa el número de secuencia de cada segmento para saber dónde encajan los datos del segmento dentro del flujo. En cualquier momento, un receptor habrá reconstruido cero o más octetos de manera contigua desde el comienzo del flujo, pero también puede tener piezas adicionales del flujo provenientes de segmentos que llegaron fuera de orden. El receptor siempre confirma el prefijo contiguo más largo del flujo que ha sido recibido correctamente. Cada confirmación especifica un valor de secuencia uno mayor que la posición de octeto más alta dentro del prefijo contiguo que recibió. Así, el emisor recibe retroalimentación continua del receptor a medida que avanza a través del flujo. Podemos resumir esta idea importante: una confirmación TCP especifica el número de secuencia del siguiente octeto que el receptor espera recibir.

El esquema de confirmación de TCP se llama acumulativo (cumulative) porque informa cuánto del flujo se ha acumulado. Las confirmaciones acumulativas tienen tanto ventajas como desventajas. Una ventaja es que las confirmaciones son fáciles de generar y no ambiguas. Otra ventaja es que las confirmaciones perdidas no necesariamente obligan a la retransmisión. Una desventaja importante es que el emisor no recibe información sobre todas las transmisiones exitosas, sino solo sobre una única posición en el flujo que ha sido recibida. Para entender por qué la falta de información sobre todas las transmisiones exitosas hace que las confirmaciones acumulativas sean menos eficientes, pensemos en una ventana que abarca 5000 octetos comenzando en la posición 101 del flujo, y supongamos que el emisor ha transmitido todos los datos de la ventana enviando cinco segmentos. Supongamos además que el primer segmento se pierde, pero todos los demás llegan intactos. A medida que llega cada segmento, el receptor envía una confirmación, pero cada confirmación especifica el octeto 101, el siguiente octeto contiguo más alto que espera recibir. No hay forma de que el receptor le indique al emisor que la mayor parte de los datos de la ventana actual ya han

llegado. En nuestro ejemplo, cuando ocurre un timeout en el lado del emisor, este debe elegir entre dos esquemas potencialmente ineficientes. Puede optar por retransmitir un solo segmento o los cinco segmentos. Retransmitir los cinco segmentos es ineficiente. Cuando llegue el primer segmento, el receptor tendrá todos los datos de la ventana y confirmará 5101. Si el emisor sigue el estándar aceptado y retransmite solo el primer segmento sin confirmar, debe esperar la confirmación antes de poder decidir qué y cuánto enviar. Así, la retransmisión vuelve al paradigma de enviar-y-esperar, perdiendo las ventajas de tener una ventana grande.

Una de las ideas más importantes y complejas de TCP está incorporada en la forma en que maneja el timeout y la retransmisión. Al igual que otros protocolos confiables, TCP espera que el destino envíe confirmaciones cada vez que recibe con éxito nuevos octetos del flujo de datos. Cada vez que envía un segmento, TCP inicia un temporizador y espera una confirmación. Si el temporizador expira antes de que los datos del segmento hayan sido confirmados, TCP asume que el segmento se perdió o se corrompió, y lo retransmite.

Para entender por qué el algoritmo de retransmisión de TCP difiere del algoritmo usado en muchos otros protocolos de red, debemos recordar que TCP está pensado para usarse en un entorno de internet. En una internet, un segmento que viaja entre un par de máquinas puede atravesar una única red de área local (por ejemplo, una Ethernet de alta velocidad), o puede viajar a través de múltiples redes intermedias mediante múltiples enrutadores. Así, es imposible saber a priori qué tan rápido regresarán las confirmaciones a la fuente. Además, el retraso en cada enrutador depende del tráfico, por lo que el tiempo total requerido para que un segmento viaje al destino y una confirmación regrese a la fuente puede variar dramáticamente de un instante a otro.

Para comprender esto, consideremos el entorno para el cual fue diseñado TCP. La figura 11.11 ilustra la situación mostrando mediciones de tiempos de ida y vuelta (RTT, round trip times) para 100 paquetes consecutivos enviados a través de la Internet global de la década de 1980.

Aunque la mayoría de las redes modernas no se comportan tan mal, el gráfico ilustra las situaciones que TCP está diseñado para acomodar: retrasos increíblemente largos y cambios en el retraso de ida y vuelta en una conexión dada. Para manejar la situación, TCP usa un algoritmo de retransmisión adaptativo. Es decir, TCP supervisa el tiempo de ida y vuelta en cada conexión y calcula valores razonables para los timeouts. A medida que cambia el rendimiento de una conexión, TCP revisa su valor de timeout (es decir, se adapta al cambio). Para recolectar los datos necesarios para un algoritmo adaptativo, TCP registra el momento en que se envía cada segmento y el momento en que llega una confirmación para los datos de

ese segmento. A partir de los dos momentos, TCP calcula un tiempo transcurrido conocido como muestra de tiempo de ida y vuelta (round trip sample). Cada vez que obtiene una nueva muestra de tiempo de ida y vuelta, TCP debe ajustar su noción del tiempo promedio de ida y vuelta para la conexión. Para hacerlo, TCP usa un promedio ponderado para estimar el tiempo de ida y vuelta, y usa cada nueva muestra de tiempo de ida y vuelta para actualizar el promedio. La técnica de promediado original usaba un factor de ponderación constante, α , donde $0 \leq \alpha < 1$, para ponderar el promedio anterior frente a la última muestra de tiempo de ida y vuelta:

$$RTT = \alpha \times RTT + (1 - \alpha) \times \text{New_Round_Trip_Sample}$$

La idea es que elegir un valor de α cercano a 1 hace que el promedio ponderado sea inmune a cambios que duran poco tiempo (por ejemplo, un solo segmento que experimenta un retraso largo). Elegir un valor de α cercano a 0 hace que el promedio ponderado responda muy rápidamente a los cambios en el retraso. Cuando envía un segmento, TCP calcula un valor de timeout en función de la estimación actual del tiempo de ida y vuelta. Las primeras implementaciones de TCP usaban un factor de ponderación constante, β ($\beta > 1$), y hacían que el timeout fuera mayor que el promedio actual de tiempo de ida y vuelta:

$$\text{Timeout} = \beta \times RTT$$

Podemos resumir las ideas presentadas hasta ahora: para adaptarse a los retrasos variables encontrados en un entorno de internet, TCP usa un algoritmo de retransmisión adaptativo que supervisa los retrasos en cada conexión y ajusta su parámetro de timeout en consecuencia.

Medición precisa de las muestras de tiempo de ida y vuelta

En teoría, medir una muestra de tiempo de ida y vuelta es trivial: consiste en restar el momento en que se envía el segmento del momento en que llega la confirmación. Sin embargo, surgen complicaciones porque TCP usa un esquema de confirmación acumulativa en el que una confirmación se refiere a los datos recibidos, y no a la instancia de un datagrama específico que transportó los datos.

Consideremos una retransmisión. TCP forma un segmento, lo coloca en un datagrama y lo envía; el temporizador expira, y TCP envía el segmento nuevamente en un segundo

datagrama. Debido a que ambos datagramas transportan exactamente los mismos datos de segmento, el emisor no tiene forma de saber si una confirmación corresponde al datagrama original o al retransmitido. Este fenómeno se ha llamado ambigüedad de confirmación (acknowledgement ambiguity). ¿Debería TCP asumir que una confirmación corresponde a la transmisión más temprana (es decir, la original) o a la más reciente (es decir, la última retransmisión)?

Sorprendentemente, ninguna de las dos suposiciones funciona. Asociar la confirmación con la transmisión original puede hacer que el tiempo de ida y vuelta estimado crezca sin límite en los casos en que una internet pierde datagramas. Si una confirmación llega después de una o más retransmisiones, TCP medirá la muestra de tiempo de ida y vuelta desde la transmisión original, y calculará un nuevo RTT usando la muestra excesivamente larga. Así, el RTT crecerá ligeramente. La próxima vez que TCP envíe un segmento, el RTT mayor resultará en timeouts ligeramente más largos, de modo que si una confirmación llega después de una o más retransmisiones, la siguiente muestra de tiempo de ida y vuelta será aún mayor, y así sucesivamente.

Asociar la confirmación con la retransmisión más reciente también puede fallar. Consideremos qué sucede cuando el retraso extremo a extremo aumenta repentinamente. Cuando TCP envía un segmento, usa la antigua estimación del tiempo de ida y vuelta para calcular un timeout, que ahora resulta demasiado pequeño. El segmento llega y una confirmación comienza a regresar, pero el aumento en el retraso hace que el temporizador expire antes de que llegue la confirmación, por lo que TCP retransmite el segmento. Poco después de que TCP retransmite, llega la primera confirmación y se asocia con la retransmisión. La muestra de tiempo de ida y vuelta será demasiado pequeña y resultará en una ligera disminución del tiempo de ida y vuelta estimado, RTT. Desafortunadamente, reducir el tiempo de ida y vuelta estimado garantiza que TCP establecerá un timeout demasiado pequeño para el siguiente segmento. En última instancia, el tiempo de ida y vuelta estimado puede estabilizarse en un valor, T , tal que el tiempo de ida y vuelta correcto sea ligeramente mayor que algún múltiplo de T . Se observaron implementaciones de TCP que asociaban las confirmaciones con la retransmisión más reciente en un estado estable con un RTT ligeramente menor a la mitad del valor correcto (es decir, TCP envía cada segmento exactamente dos veces, aunque no ocurra pérdida alguna).

Algoritmo de Karn y retroceso del temporizador (timer backoff)

Si ni la transmisión original ni la transmisión más reciente proporcionan tiempos de ida y vuelta precisos, ¿qué debería hacer TCP? La respuesta aceptada es simple: TCP no debería actualizar la estimación del tiempo de ida y vuelta para los segmentos retransmitidos. La idea, conocida como Algoritmo de Karn (Karn's Algorithm), evita por completo el problema de las confirmaciones ambiguas al ajustar el tiempo de ida y vuelta estimado únicamente para confirmaciones no ambiguas (confirmaciones que llegan para segmentos que solo han sido transmitidos una vez).

Por supuesto, una implementación simplista del algoritmo de Karn, que simplemente ignora los tiempos de los segmentos retransmitidos, también puede llevar a un fallo. Consideremos qué sucede cuando TCP envía un segmento después de un aumento brusco en el retraso. TCP calcula un timeout usando la estimación existente del tiempo de ida y vuelta. El timeout será demasiado pequeño para el nuevo retraso y forzará una retransmisión. Si TCP ignora las confirmaciones de los segmentos retransmitidos, nunca actualizará la estimación y el ciclo continuará.

Para hacer frente a este tipo de fallos, el algoritmo de Karn exige que el emisor combine los timeouts de retransmisión con una estrategia de retroceso del temporizador (timer backoff). La técnica de retroceso calcula un timeout inicial usando una fórmula como la mostrada anteriormente. Sin embargo, si el temporizador expira y provoca una retransmisión, TCP aumenta el timeout. De hecho, cada vez que debe retransmitir un segmento, TCP aumenta el timeout (para evitar que los timeouts se vuelvan ridículamente largos, la mayoría de las implementaciones limitan los aumentos a un límite superior que es mayor que el retraso a lo largo de cualquier ruta en la internet). Las implementaciones usan una variedad de técnicas para calcular el retroceso. La mayoría elige un factor multiplicativo, γ , y establece el nuevo valor como:

$$\text{new_timeout} = \gamma \times \text{timeout}$$

Típicamente, γ es 2. (Se ha argumentado que valores de γ menores a 2 conducen a inestabilidades.) Otras implementaciones usan una tabla de factores multiplicativos, permitiendo un retroceso arbitrario en cada paso. El algoritmo de Karn combina la técnica de retroceso con la estimación del tiempo de ida y vuelta para resolver el problema de nunca aumentar las estimaciones del tiempo de ida y vuelta:

Al calcular la estimación del tiempo de ida y vuelta, ignorar las muestras que correspondan a segmentos retransmitidos, pero usar una estrategia de retroceso y conservar el valor de timeout de un paquete retransmitido para los paquetes subsiguientes hasta que se obtenga una muestra válida.

En términos generales, cuando una internet se comporta mal, el algoritmo de Karn separa el cálculo del valor de timeout de la estimación actual del tiempo de ida y vuelta. Usa la estimación del tiempo de ida y vuelta para calcular un valor de timeout inicial, pero luego retrocede (backs off) el timeout en cada retransmisión hasta que pueda transferir con éxito un segmento. Cuando envía segmentos subsiguientes, TCP conserva el valor de timeout resultante del retroceso. Finalmente, cuando llega una confirmación correspondiente a un segmento que no requirió retransmisión, TCP vuelve a calcular la estimación del tiempo de ida y vuelta y restablece el timeout en consecuencia. La experiencia muestra que el algoritmo de Karn funciona bien incluso en redes con alta pérdida de paquetes.

Respuesta a la congestión

Podría parecer que el software TCP podría diseñarse considerando solo la interacción entre los dos puntos finales de una conexión y los retrasos de comunicación entre esos puntos finales. En la práctica, sin embargo, TCP también debe reaccionar ante la congestión en una internet. La congestión es una condición de retraso severo causada por una sobrecarga de datagramas en uno o más puntos de conmutación (por ejemplo, en los enrutadores). Cuando ocurre congestión, los retrasos aumentan y el enrutador comienza a poner en cola los datagramas hasta que puede reenviarlos. Debemos recordar que cada enrutador tiene una capacidad de almacenamiento finita, y que los datagramas compiten por ese almacenamiento (es decir, en una internet basada en datagramas, no hay preasignación de recursos a conexiones TCP individuales). En el peor de los casos, el número total de datagramas que llegan al enrutador congestionado crece hasta que el enrutador alcanza su capacidad y comienza a descartar datagramas.

Los puntos finales normalmente no conocen los detalles de dónde ha ocurrido la congestión ni por qué. Para ellos, la congestión simplemente significa un aumento del retraso. Desafortunadamente, la mayoría de los protocolos de transporte usan timeout y retransmisión, por lo que responden al aumento del retraso retransmitiendo datagramas. Las retransmisiones agravan la congestión en lugar de aliviarla. Si no se controla, el aumento del tráfico producirá un aumento del retraso, lo que llevará a un aumento del tráfico, y así

sucesivamente, hasta que la red se vuelva inutilizable. Esta condición se conoce como colapso por congestión (congestion collapse).

TCP puede ayudar a evitar la congestión reduciendo las tasas de transmisión cuando esta ocurre. De hecho, TCP reacciona rápidamente reduciendo automáticamente la tasa de transmisión siempre que ocurren retrasos. Por supuesto, los algoritmos para evitar la congestión deben construirse cuidadosamente, porque incluso en condiciones normales de operación una internet exhibirá una amplia variación en los retrasos de ida y vuelta. Para evitar la congestión, el estándar TCP actualmente recomienda usar dos técnicas: inicio lento (slow-start) y disminución multiplicativa (multiplicative decrease). Las dos están relacionadas y pueden implementarse fácilmente.

Dijimos que, para cada conexión, TCP debe recordar el tamaño de la ventana del receptor (es decir, el tamaño de buffer anunciado en las confirmaciones). Para controlar la congestión, TCP mantiene un segundo límite, llamado tamaño de ventana de congestión o ventana de congestión (congestion window), que usa para restringir el flujo de datos a menos que el tamaño de buffer del receptor cuando ocurre congestión. Es decir, en cualquier momento, TCP actúa como si el tamaño de ventana fuera:

$$\text{Allowed_window} = \min (\text{receiver_advertisement}, \text{congestion_window})$$

En estado estable, en una conexión no congestionada, la ventana de congestión tiene el mismo tamaño que la ventana del receptor. Reducir la ventana de congestión reduce el tráfico que TCP inyectará en la conexión. Para estimar el tamaño de la ventana de congestión, TCP asume que la mayor parte de la pérdida de datagramas proviene de la congestión y usa la siguiente estrategia:

Disminución multiplicativa para evitar la congestión: al perderse un segmento, reducir la ventana de congestión a la mitad (pero nunca reducir la ventana a menos de un segmento). Al transmitir los segmentos que permanecen dentro de la ventana permitida, hacer un retroceso exponencial del temporizador de retransmisión.

Debido a que TCP reduce la ventana de congestión a la mitad por cada pérdida, disminuye la ventana exponencialmente si la pérdida continúa. En otras palabras, si es probable que haya congestión, TCP reduce el volumen de tráfico exponencialmente y la tasa de retransmisión exponencialmente. Si la pérdida continúa, TCP eventualmente limita la transmisión a un solo datagrama y continúa duplicando los valores de timeout antes de retransmitir. La idea es proporcionar una reducción de tráfico rápida y significativa para dar tiempo suficiente a los enrutadores de despejar los datagramas ya presentes en sus colas.

¿Cómo puede TCP recuperarse cuando termina la congestión? Se podría sospechar que TCP debería revertir la disminución multiplicativa y duplicar la ventana de congestión cuando el tráfico comienza a fluir nuevamente. Sin embargo, hacerlo produce un sistema inestable que oscila violentamente entre la ausencia de tráfico y la congestión. En cambio, TCP usa una técnica llamada inicio lento (slow-start) para escalar la transmisión.

Recuperación aditiva por inicio lento (Slow-Start): siempre que se inicie tráfico en una nueva conexión o se aumente el tráfico después de un período de congestión, iniciar la ventana de congestión con el tamaño de un solo segmento e incrementar la ventana de congestión en un segmento cada vez que llega una confirmación.

El inicio lento evita saturar la internet subyacente con tráfico adicional inmediatamente después de que se despeja la congestión, así como cuando se inicia una nueva conexión.

El término inicio lento puede ser un nombre inapropiado, porque en condiciones ideales el inicio no es muy lento. TCP inicializa la ventana de congestión en 1, envía un segmento inicial y espera. Cuando llega la confirmación, aumenta la ventana de congestión a 2, envía dos segmentos y espera. Cuando llegan las dos confirmaciones, cada una aumenta la ventana de congestión en 1, por lo que TCP puede enviar 4 segmentos. Las confirmaciones de esos segmentos aumentarán la ventana de congestión a 8. En cuatro tiempos de ida y vuelta, TCP puede enviar 16 segmentos, a menudo suficiente para alcanzar el límite de la ventana del receptor. Incluso para ventanas extremadamente grandes, solo se necesitan $\log_2 N$ tiempos de ida y vuelta antes de que TCP pueda enviar N segmentos.

Para evitar aumentar el tamaño de la ventana demasiado rápido y causar congestión adicional, TCP agrega una restricción adicional. Una vez que la ventana de congestión alcanza la mitad de su tamaño original antes de la congestión, TCP entra en una fase de evitación de congestión (congestion avoidance) y reduce la velocidad del incremento. Durante la evitación de congestión, aumenta la ventana de congestión en 1 solo si todos los segmentos de la ventana han sido confirmados. El enfoque general se conoce como Incremento Aditivo, Disminución Multiplicativa (AIMD, Additive Increase Multiplicative Decrease).

En conjunto, el inicio lento, el incremento aditivo, la disminución multiplicativa, la medición de la variación y el retroceso exponencial del temporizador mejoran drásticamente el rendimiento de TCP sin agregar una sobrecarga computacional significativa al software del protocolo. Las versiones de TCP que usan estas técnicas han mejorado significativamente el rendimiento respecto a versiones anteriores.

Recuperación rápida y otras modificaciones de respuesta

Se han hecho modificaciones menores a TCP a lo largo de muchos años. Una versión temprana de TCP, a veces denominada Tahoe, usaba el esquema de retransmisión descrito anteriormente, esperando a que expirara un temporizador antes de retransmitir. En 1990 apareció la versión Reno de TCP, que introdujo varios cambios, incluyendo una heurística conocida como recuperación rápida (fast recovery) o retransmisión rápida (fast retransmit), que ofrece mayor rendimiento en casos donde solo ocurre pérdida ocasional. Después de la versión Reno, los investigadores exploraron una versión Vegas.

El truco usado en la recuperación rápida surge del esquema de confirmación acumulativa de TCP: la pérdida de un solo segmento significa que la llegada de los segmentos subsiguientes hará que el receptor genere un ACK para el punto en el flujo donde comienza el segmento faltante. Desde el punto de vista del emisor, un paquete perdido significa que llegarán múltiples confirmaciones que llevan cada una el mismo número de secuencia. La heurística de retransmisión rápida usa una serie de tres confirmaciones duplicadas (es decir, una original más tres copias idénticas) para desencadenar una retransmisión sin esperar a que expire el temporizador. En un caso donde solo se pierde un segmento, esperar a que el segmento retransmitido sea confirmado también reduce el rendimiento. Por lo tanto, para mantener un rendimiento más alto, la heurística de retransmisión rápida continúa enviando datos de la ventana mientras espera la confirmación del segmento retransmitido. Además, la ventana de congestión se infla artificialmente: la ventana de congestión se reduce a la mitad para la retransmisión, pero luego se incrementa en un segmento de tamaño máximo por cada ACK duplicado que haya llegado previamente o que llegue después de que ocurra la retransmisión. Como resultado, mientras ocurre la retransmisión rápida, TCP mantiene muchos segmentos "en vuelo" (in flight) entre el emisor y el receptor.

Una optimización adicional de la heurística de retransmisión rápida se incorporó en una modificación posterior de TCP conocida como la versión NewReno. La optimización maneja un caso en el que se pierden dos segmentos dentro de una sola ventana. En esencia, cuando ocurre la retransmisión rápida, NewReno registra información sobre la ventana actual y retransmite como se describió anteriormente. Cuando llega el ACK para el segmento retransmitido, hay dos posibilidades: el ACK especifica el número de secuencia al final de la ventana (en cuyo caso el segmento retransmitido era el único que faltaba en la ventana), o el ACK especifica un número de secuencia mayor que el del segmento faltante, pero menor que el final de la ventana (en cuyo caso también se ha perdido un segundo segmento de la ventana). En este último caso, NewReno procede a retransmitir el segundo segmento faltante. Se han propuesto y usado modificaciones menores al esquema AIMD en versiones

posteriores de TCP. Para entenderlas, consideremos cómo AIMD cambia la ventana de congestión del emisor, w , en respuesta a la pérdida de un segmento o a la llegada de una confirmación:

$$w \leftarrow w - aw \quad \text{when loss is detected}$$

$$w \leftarrow w + \frac{b}{w} \quad \text{when an ACK arrives}$$

En el esquema original, a es .5 y b es 1. Al pensar en protocolos como STCP, usado en redes de sensores, los investigadores propusieron establecer a en 0.125 y b en 0.01 para evitar que la ventana de congestión oscile y aumentar ligeramente el rendimiento. Otras propuestas de modificación (por ejemplo, un protocolo conocido como HSTCP) sugieren hacer que a y b sean funciones de w (es decir, $a(w)$ y $b(w)$). Finalmente, propuestas de control de congestión de TCP como Vegas y FAST usan el aumento del RTT como medida de congestión en lugar de la pérdida de paquetes, y definen el tamaño de la ventana de congestión como una función del RTT medido. Típicamente, las modificaciones solo producen mejoras de rendimiento en casos especiales (por ejemplo, redes con alto ancho de banda y bajas tasas de pérdida); el control de congestión AIMD de NewReno se usa para los casos generales.

Una propuesta final relacionada con el control de congestión de TCP concierne a UDP. Obsérvese que, aunque TCP reduce la transmisión cuando ocurre congestión, UDP no lo hace, lo que significa que a medida que los flujos TCP continúan retrocediendo, los flujos UDP consumen más del ancho de banda. Se propuso una solución conocida como Control de Tasa Amigable con TCP (TFRC, TCP Friendly Rate Control). TFRC intenta emular el comportamiento de TCP haciendo que un receptor UDP reporte la pérdida de datagramas de vuelta al emisor, y haciendo que el emisor use la pérdida reportada para calcular una tasa a la que deben enviarse los datagramas UDP; TFRC solo ha sido adoptado para casos especiales.

Mecanismos de retroalimentación explícita (SACK y ECN)

La mayoría de las versiones de TCP usan técnicas implícitas para detectar pérdida y congestión. Es decir, TCP usa el timeout y los ACK duplicados para detectar pérdida, y los cambios en los tiempos de ida y vuelta para detectar congestión. Los investigadores han observado que son posibles ligeras mejoras si TCP incluye mecanismos que proporcionen

dicha información de manera explícita. Las siguientes dos secciones describen dos técnicas explícitas que han sido propuestas.

1 Confirmación selectiva (SACK)

La alternativa al mecanismo de confirmación acumulativa de TCP se conoce como mecanismo de confirmación selectiva. En esencia, las confirmaciones selectivas permiten que un receptor especifique exactamente qué datos han sido recibidos y cuáles siguen faltando. La principal ventaja de las confirmaciones selectivas surge en situaciones donde ocurre una pérdida ocasional: las confirmaciones selectivas permiten que un emisor sepa exactamente qué segmentos retransmitir.

El mecanismo de Confirmación Selectiva (SACK, Selective ACKnowledgement) propuesto para TCP no reemplaza por completo el mecanismo de confirmación acumulativa, ni es obligatorio. En cambio, TCP incluye dos opciones para SACK. La primera opción se usa cuando se establece la conexión, para permitir que un emisor especifique que SACK está permitido. La segunda opción es usada por un receptor al enviar una confirmación, para incluir información sobre bloques específicos de datos que fueron recibidos. La información de cada bloque incluye el primer número de secuencia de un bloque (llamado borde izquierdo, left edge) y el número de secuencia inmediatamente posterior al bloque (llamado borde derecho, right edge). Debido a que el tamaño máximo del encabezado de un segmento es fijo, una confirmación puede contener como máximo cuatro bloques SACK. Curiosamente, los documentos de SACK no especifican exactamente cómo debe responder un emisor a SACK; la mayoría de las implementaciones retransmiten todos los bloques faltantes.

2 Notificación explícita de congestión

Una segunda técnica propuesta para evitar la medición implícita está pensada para manejar la congestión en la red. Conocida como Notificación Explícita de Congestión (ECN, Explicit Congestion Notification), el mecanismo requiere que los enrutadores a lo largo de una internet notifiquen a TCP a medida que ocurre la congestión. El mecanismo es conceptualmente sencillo: mientras un segmento TCP atraviesa la internet, los enrutadores a lo largo de la ruta usan un par de bits en el encabezado IP para registrar la congestión.

Así, cuando llega un segmento, el receptor sabe si el segmento experimentó congestión en algún punto. Desafortunadamente, es el emisor, no el receptor, quien necesita conocer la congestión. Por lo tanto, el receptor usa el siguiente ACK para informar al emisor que ocurrió congestión. El emisor entonces responde reduciendo su ventana de congestión. ECN usa dos

bits en el encabezado IP para permitir que los enrutadores registren la congestión, y usa dos bits en el encabezado TCP (tomados del área reservada) para permitir que el TCP emisor y el receptor se comuniquen. Uno de los bits del encabezado TCP es usado por un receptor para enviar información de congestión de vuelta a un emisor; el otro bit permite que un emisor informe al receptor que la notificación de congestión ha sido recibida. Los bits en el encabezado IP se toman de bits no usados del campo TYPE OF SERVICE. Un enrutador puede optar por establecer cualquiera de los dos bits para especificar que ocurrió congestión (se usan dos bits para hacer el mecanismo más robusto).

Congestión, tail drop y TCP

Dijimos que los protocolos de comunicación se dividen en capas para permitir que los diseñadores se centren en un solo problema a la vez. La separación de funcionalidad en capas es a la vez necesaria y útil, significa que una capa puede cambiarse sin afectar otras capas, pero también significa que las capas operan de manera aislada. Por ejemplo, debido a que opera de extremo a extremo, TCP permanece sin cambios cuando cambia la ruta entre los puntos finales (por ejemplo, cuando cambian las rutas o se agregan enrutadores de red adicionales). Sin embargo, el aislamiento de las capas restringe la comunicación entre capas. En particular, aunque el TCP en la fuente original interactúa con el TCP en el destino final, no puede interactuar con los elementos de capas inferiores a lo largo de la ruta. Así, ni TCP emisor ni el receptor reciben informes sobre las condiciones en la red, ni tampoco ninguno de los dos extremos informa a las capas inferiores a lo largo de la ruta antes de transferir datos.

Los investigadores han observado que la falta de comunicación entre capas significa que la elección de política o implementación en una capa puede tener un efecto dramático en el rendimiento de las capas superiores. En el caso de TCP, las políticas que los enrutadores usan para manejar los datagramas pueden tener un efecto significativo tanto en el rendimiento de una sola conexión TCP como en el rendimiento agregado de todas las conexiones. Por ejemplo, si un enrutador retrasa algunos datagramas más que otros, TCP retrocederá su temporizador de retransmisión. Si el retraso excede el timeout de retransmisión, TCP asumirá que ha ocurrido congestión. Así, aunque cada capa se define de manera independiente, los investigadores intentan diseñar mecanismos e implementaciones que funcionen bien con los protocolos de otras capas.

La interacción más importante entre las políticas de implementación de IP y TCP ocurre cuando un enrutador se sobrecarga y descarta datagramas. Debido a que un enrutador coloca cada datagrama entrante en una cola en memoria hasta que puede procesarlo, la política se

centra en la gestión de la cola. Cuando los datagramas llegan más rápido de lo que pueden reenviarse, la cola crece; cuando los datagramas llegan más lento de lo que pueden reenviarse, la cola se reduce. Sin embargo, debido a que la memoria es finita, la cola no puede crecer sin límite. Los primeros enrutadores usaban una política de descarte por la cola (tail-drop) para manejar el desbordamiento de la cola:

Política de tail-drop para enrutadores: si una cola de paquetes está llena cuando debe colocarse un datagrama en ella, descartar el datagrama.

El nombre tail-drop surge del efecto de la política sobre una secuencia de datagramas que llega. Una vez que la cola se llena, el enrutador comienza a descartar todos los datagramas adicionales. Es decir, el enrutador descarta la "cola" (tail) de la secuencia. El tail-drop tiene un efecto interesante sobre TCP. En el caso simple en que los datagramas que atraviesan un enrutador transportan segmentos de una sola conexión TCP, la pérdida hace que TCP entre en inicio lento (slow-start), lo que reduce el rendimiento hasta que TCP comienza a recibir ACKs y aumenta la ventana de congestión. Sin embargo, puede ocurrir un problema más severo cuando los datagramas que atraviesan un enrutador transportan segmentos de muchas conexiones TCP, porque el tail-drop puede causar sincronización global. Para ver por qué, observemos que los datagramas normalmente se multiplexan, con datagramas sucesivos provenientes cada uno de una fuente distinta. Así, una política de tail-drop hace probable que el enrutador descarte un segmento de N conexiones en lugar de N segmentos de una sola conexión. La pérdida simultánea hace que las N instancias de TCP entren en inicio lento al mismo tiempo .

Detección temprana aleatoria (RED)

¿Cómo puede un enrutador evitar la sincronización global? La respuesta está en un esquema ingenioso que evita el tail-drop siempre que sea posible. Conocido como Detección Temprana Aleatoria, Descarte Temprano Aleatorio o Rechazo Temprano Aleatorio, el esquema se conoce con mayor frecuencia por su acrónimo, RED (Random Early Detection). La idea general detrás de RED reside en la aleatorización: en lugar de esperar hasta que una cola se llene por completo, un enrutador monitorea el tamaño de la cola. A medida que la cola comienza a llenarse, el enrutador elige datagramas al azar para descartarlos.

Un router que implementa RED ejecuta el algoritmo en cada cola (por ejemplo, en cada conexión de red). Para simplificar nuestra descripción, solo discutiremos una sola cola, y se asume que el lector comprende que la misma técnica debe aplicarse a las demás colas. Un enrutador usa dos valores umbral para marcar posiciones en la cola: T_{min} y T_{max} . El

funcionamiento general de RED puede describirse mediante tres reglas que determinan la disposición de un datagrama que debe colocarse en la cola:

Si la cola actualmente contiene menos de T_{min} datagramas, agregar el nuevo datagrama a la cola. Si la cola contiene más de T_{max} datagramas, descartar el nuevo datagrama. Si la cola contiene entre T_{min} y T_{max} datagramas, descartar aleatoriamente el datagrama con una probabilidad, p , que depende del tamaño actual de la cola.

La aleatoriedad de RED significa que, en lugar de esperar hasta que la cola se desborde y luego llevar a muchas conexiones TCP a inicio lento, un enrutador descarta datagramas de forma lenta y aleatoria a medida que aumenta la congestión. Podemos resumir:

Política RED para enrutadores: si la cola de entrada está llena cuando llega un datagrama, descartar el datagrama; si la cola de entrada está por debajo de un umbral mínimo, agregar el datagrama a la cola; de lo contrario, descartar el datagrama con una probabilidad que depende del tamaño de la cola.

La clave para que RED funcione bien está en la elección de los umbrales T_{min} y T_{max} y de la probabilidad de descarte p . T_{min} debe ser suficientemente grande para asegurar que la cola tenga un rendimiento suficientemente alto. Por ejemplo, si la cola está conectada a un enlace de salida, la cola debe mantener la red a una alta utilización. Además, debido a que RED opera como tail-drop cuando el tamaño de la cola excede T_{max} , el valor de T_{max} debe ser mayor que T_{min} en más del aumento típico del tamaño de la cola durante un tiempo de ida y vuelta de TCP (por ejemplo, establecer T_{max} al menos el doble de T_{min}). De lo contrario, RED puede causar las mismas oscilaciones globales que el tail-drop (por ejemplo, T_{min} puede establecerse en la mitad de T_{max}).

El cálculo de la probabilidad de descarte, p , es el aspecto más complejo de RED. En lugar de usar una constante, se calcula un nuevo valor de p para cada datagrama; el valor depende de la relación entre el tamaño actual de la cola y los umbrales. Para entender el esquema, observemos que todo el procesamiento de RED puede verse de manera probabilística. Cuando el tamaño de la cola es menor que T_{min} , RED no descarta ningún datagrama, haciendo que la probabilidad de descarte sea 0. De manera similar, cuando el tamaño de la cola es mayor que T_{max} , RED descarta todos los datagramas, haciendo que la probabilidad de descarte sea 1. Para valores intermedios del tamaño de la cola (es decir, entre T_{min} y T_{max}), la probabilidad puede variar de 0 a 1 de manera lineal. Aunque el esquema lineal forma la base del cálculo de probabilidad de RED, debe hacerse un cambio para evitar una reacción excesiva. La necesidad de este cambio surge porque el tráfico de red es a ráfagas (bursty), lo que resulta en fluctuaciones rápidas en la cola de un enrutador. Si RED usara un

esquema lineal simplista, los datagramas posteriores de cada ráfaga recibirían una alta probabilidad de ser descartados (porque llegan cuando la cola tiene más entradas). Sin embargo, un enrutador no debería descartar datagramas innecesariamente, porque hacerlo tiene un impacto negativo en el rendimiento de TCP. Así, si una ráfaga es corta, no es conveniente descartar datagramas porque la cola nunca se desbordará. Por supuesto, RED no puede posponer el descarte indefinidamente, porque una ráfaga de larga duración desbordará la cola, dando lugar a una política de tail-drop, que tiene el potencial de causar problemas de sincronización global.

¿Cómo puede RED asignar una probabilidad de descarte mayor a medida que la cola se llena, sin descartar datagramas de cada ráfaga? La respuesta está en una técnica tomada prestada de TCP: en lugar de usar el tamaño real de la cola en un instante dado, RED calcula un tamaño de cola promedio ponderado, avg, y usa el tamaño promedio para determinar la probabilidad. El valor de avg es un promedio ponderado exponencial, actualizado cada vez que llega un datagrama según la ecuación:

$$\text{avg} = (1 - \gamma) \times \text{Old_avg} + \gamma \times \text{Current_queue_size}$$

donde γ denota un valor entre 0 y 1. Si γ es suficientemente pequeño, el promedio seguirá las tendencias de largo plazo, pero permanecerá inmune a las ráfagas cortas. Además de las ecuaciones que determinan γ , RED contiene otros detalles que hemos omitido. Por ejemplo, los cálculos de RED pueden hacerse extremadamente eficientes eligiendo constantes como potencias de dos y usando aritmética de enteros.

Otro detalle importante concierne a la medición del tamaño de la cola, lo cual afecta tanto el cálculo de RED como su efecto general sobre TCP. En particular, debido a que el tiempo requerido para reenviar un datagrama es proporcional a su tamaño, tiene sentido medir la cola en octetos en lugar de en datagramas; hacerlo requiere solo cambios menores en las ecuaciones para p y γ . Medir el tamaño de la cola en octetos afecta el tipo de tráfico que se descarta, porque hace que la probabilidad de descarte sea proporcional a la cantidad de datos que un emisor pone en el flujo, en lugar del número de segmentos. Los datagramas pequeños (por ejemplo, los que transportan tráfico de inicio de sesión remoto o solicitudes a servidores) tienen una menor probabilidad de ser descartados que los datagramas grandes (por ejemplo, los que transportan tráfico de transferencia de archivos).

Una consecuencia positiva de usar el tamaño del datagrama es que, cuando las confirmaciones viajan a través de una ruta congestionada, tienen una menor probabilidad de

ser descartadas. Como resultado, si llega un segmento de datos (grande), el TCP emisor recibirá el ACK y evitará una retransmisión innecesaria.

Tanto el análisis como las simulaciones muestran que RED funciona bien. Maneja la congestión, evita la sincronización que resulta del tail-drop, y permite ráfagas cortas sin descartar datagramas innecesariamente. En consecuencia, el IETF ahora recomienda que los enrutadores implementen RED.

Establecimiento de una conexión TCP

Para establecer una conexión, TCP usa un protocolo de tres pasos (three-way handshake). Es decir, se intercambian tres mensajes que permiten que cada lado acuerde formar una conexión y sepa que el otro lado también ha aceptado. El primer segmento de un handshake puede identificarse porque tiene activado el bit SYN en el campo de código. El segundo mensaje tiene activados tanto el bit SYN como el bit ACK, para indicar que confirma el primer segmento SYN y continúa el handshake. El mensaje final del handshake es solo una confirmación, y se usa únicamente para informar al destino que ambos lados están de acuerdo en que se ha establecido una conexión.

Normalmente, el software TCP en una máquina espera pasivamente el handshake, y el software TCP en otra máquina lo inicia. Sin embargo, el handshake está diseñado cuidadosamente para funcionar incluso si ambas máquinas intentan iniciar una conexión simultáneamente. Así, una conexión puede establecerse desde cualquiera de los dos extremos, o desde ambos extremos simultáneamente. Una vez que se ha establecido la conexión, los datos pueden fluir en ambas direcciones igualmente bien (es decir, la conexión es simétrica). Es decir, no hay un maestro ni un esclavo, y el lado que inicia la conexión no tiene habilidades ni privilegios especiales. En el caso más simple, el handshake procede como muestra la figura 11.14.

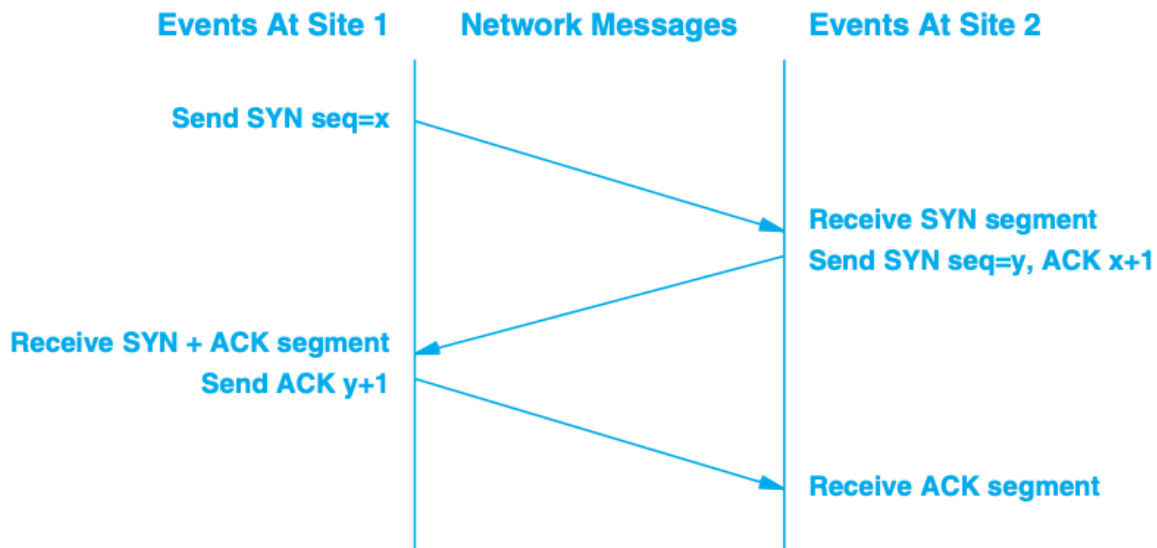


Figure 11.14 The sequence of messages in a three-way handshake. Time proceeds down the page; diagonal lines represent segments sent between sites.

Podría parecer que un intercambio de dos mensajes sería suficiente para establecer una conexión. Sin embargo, el handshake de tres pasos es tanto necesario como suficiente para lograr una sincronización correcta entre los dos extremos de la conexión, dada la semántica de entrega de internet. Para entender por qué el establecimiento de conexiones es difícil, recordemos que TCP usa un servicio de entrega de paquetes no confiable. Por lo tanto, los mensajes pueden perderse, retrasarse, duplicarse o entregarse fuera de orden. Para lidiar con la pérdida, TCP debe retransmitir las solicitudes. Sin embargo, pueden surgir problemas si un retraso excesivo provoca una retransmisión, lo que significa que tanto la copia original como la retransmitida llegan mientras se está estableciendo la conexión. ¡Las solicitudes retransmitidas incluso pueden retrasarse hasta después de que una conexión haya sido establecida, usada y terminada! El handshake de tres pasos y las reglas que impiden reiniciar una conexión después de que ha terminado están cuidadosamente diseñadas para compensar todas las situaciones posibles.

Números de secuencia iniciales

El handshake de tres pasos cumple dos funciones importantes. Garantiza que ambos lados estén listos para transferir datos (y que sepan que ambos están listos), y permite que ambos lados acuerden números de secuencia iniciales. Los números de secuencia se envían y confirman durante el handshake. Cada máquina debe elegir al azar un número de secuencia inicial que usará para identificar los octetos en el flujo que está enviando. Los números de

secuencia no siempre pueden comenzar con el mismo valor. En particular, TCP no puede simplemente elegir la secuencia 1 cada vez que crea una conexión (uno de los ejercicios examina los problemas que pueden surgir si lo hiciera). Por supuesto, es importante que ambos lados acuerden un número inicial, de modo que los números de octeto usados en las confirmaciones coincidan con los usados en los segmentos de datos.

Para ver cómo las máquinas pueden acordar los números de secuencia para dos flujos después de solo tres mensajes, recordemos que cada segmento contiene tanto un campo de número de secuencia como un campo de confirmación. La máquina que inicia un handshake, llamémosla A, pasa su número de secuencia inicial, x , en el campo de secuencia del primer segmento SYN del handshake de tres pasos. La otra máquina, B, recibe el SYN, registra el número de secuencia, y responde enviando su propio número de secuencia inicial en el campo de secuencia, junto con una confirmación que especifica que B espera el octeto $x+1$. En el mensaje final del handshake, A "confirma" haber recibido de B todos los octetos hasta y . En todos los casos, las confirmaciones siguen la convención de usar el número del siguiente octeto esperado.

Hemos descrito cómo TCP normalmente lleva a cabo el handshake de tres pasos intercambiando segmentos que contienen una cantidad mínima de información. Debido al diseño del protocolo, es posible enviar datos junto con los números de secuencia iniciales en los segmentos del handshake. En tales casos, el software TCP debe retener los datos hasta que el handshake se complete. Una vez que se ha establecido una conexión, el software TCP puede liberar los datos retenidos y entregarlos rápidamente a un programa de aplicación en espera. Se remite al lector a la especificación del protocolo para más detalles.

Cierre de una conexión TCP

Dos aplicaciones que usan TCP para comunicarse pueden terminar la conversación de manera ordenada usando la operación close. Nuevamente, es importante que ambos lados acuerden cerrar una conexión y que ambos sepan que la conexión está cerrada. Por lo tanto, TCP usa un handshake de tres pasos para cerrar conexiones. Para entender el handshake usado para cerrar una conexión, recordemos que las conexiones TCP son full duplex, y que las consideramos como conteniendo dos transferencias de flujo independientes, una en cada dirección. Cuando un programa de aplicación le indica a TCP que ya no tiene más datos para enviar, TCP cerrará la conexión en esa dirección.

Para cerrar su mitad de una conexión, el TCP emisor termina de transmitir los datos restantes, espera a que el receptor los confirme, y luego envía un segmento con el bit FIN activado. Al

Optativa II - Integración de Redes (Intemetworking) – 2026

Teoría

recibir un FIN, TCP envía una confirmación y luego informa a la aplicación que el otro lado ha terminado de enviar datos. Los detalles dependen del sistema operativo, pero la mayoría de los sistemas usan el mecanismo de "fin de archivo" (end-of-file). Una vez que se ha cerrado una conexión en una dirección dada, TCP se niega a aceptar más datos para esa dirección. Mientras tanto, los datos pueden seguir fluyendo en la dirección opuesta hasta que el emisor la cierre. Por supuesto, un punto final TCP que todavía está recibiendo datos debe enviar confirmaciones, incluso si la transmisión de datos en la dirección inversa ha terminado. Cuando ambas direcciones se han cerrado, el software TCP en cada punto final elimina su registro de la conexión.

Los detalles del cierre de una conexión son incluso más sutiles de lo sugerido anteriormente, porque TCP usa un handshake de tres pasos modificado para cerrar una conexión. La figura 11.15 ilustra los mensajes que se intercambian en el caso típico en que toda la comunicación ha finalizado y la conexión se cierra en ambas direcciones.

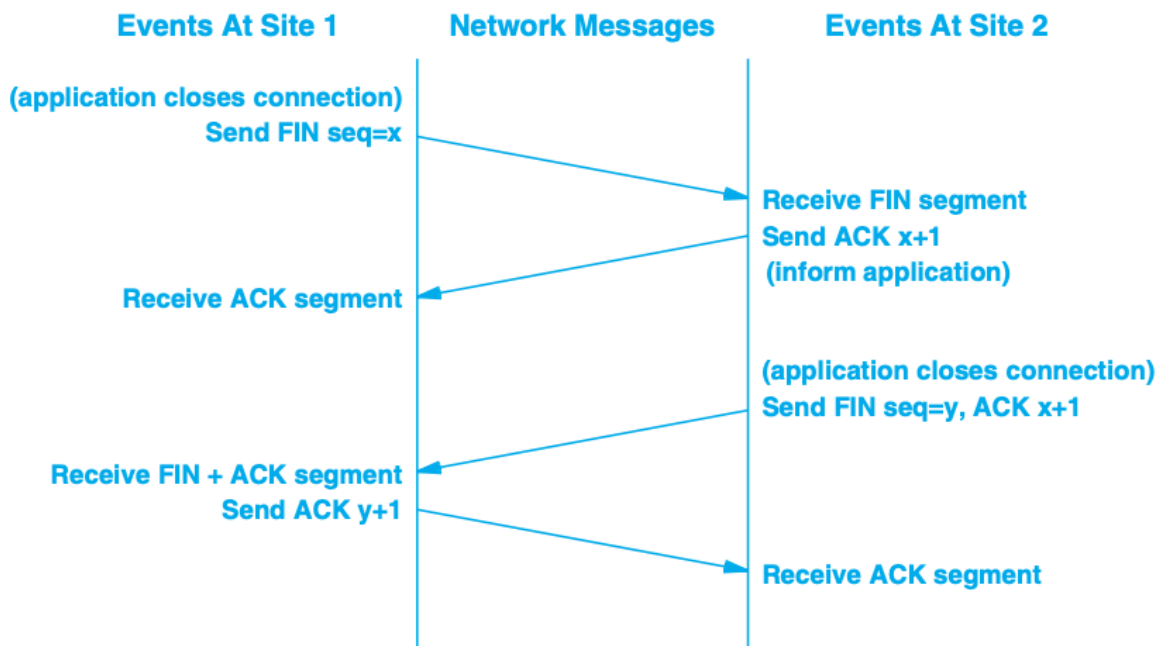


Figure 11.15 The three-way handshake used to close a connection with an extra ACK sent immediately upon receipt of a FIN.

La diferencia entre los handshakes de tres pasos usados para establecer y cerrar conexiones ocurre después de que una máquina recibe el segmento FIN inicial. En lugar de generar un segundo segmento FIN de inmediato, TCP envía una confirmación y luego informa a la aplicación de la solicitud de cierre. Informar al programa de aplicación de la solicitud y obtener una respuesta puede tomar un tiempo considerable (por ejemplo, puede involucrar

interacción humana). La confirmación evita la retransmisión del segmento FIN inicial durante ese retraso. Finalmente, cuando el programa de aplicación indica a TCP que cierre la conexión por completo, TCP envía el segundo segmento FIN, y el sitio original responde con el tercer mensaje, un ACK.

Reinicio de una conexión TCP

Normalmente, un programa de aplicación usa la operación close para cerrar una conexión cuando termina de enviar datos. Así, cerrar conexiones se considera una parte normal de su uso, análoga a cerrar archivos. Decimos que la conexión terminó de manera ordenada (gracefully). Sin embargo, a veces surgen condiciones anormales que obligan a una aplicación o al software de red a interrumpir una conexión sin un cierre ordenado. TCP proporciona un mecanismo de reinicio (reset) para manejar desconexiones anormales. Para reiniciar una conexión, un lado inicia la terminación enviando un segmento con el bit RST (RESET) activado en el campo CODE. El otro lado responde a un segmento de reinicio de inmediato, abortando la conexión. Cuando ocurre un reinicio, TCP informa a cualquier aplicación local que estuviera usando la conexión. Nótese que un reinicio ocurre de inmediato y no puede deshacerse. Lo pensamos como un aborto instantáneo que termina la transferencia en ambas direcciones y libera recursos, como los buffers.

Máquina de estados de TCP

Al igual que la mayoría de los protocolos, el funcionamiento de TCP puede explicarse mejor con un modelo teórico llamado máquina de estados finitos. La figura 11.16 muestra la máquina de estados de TCP, con círculos que representan estados y flechas que representan las transiciones entre ellos.

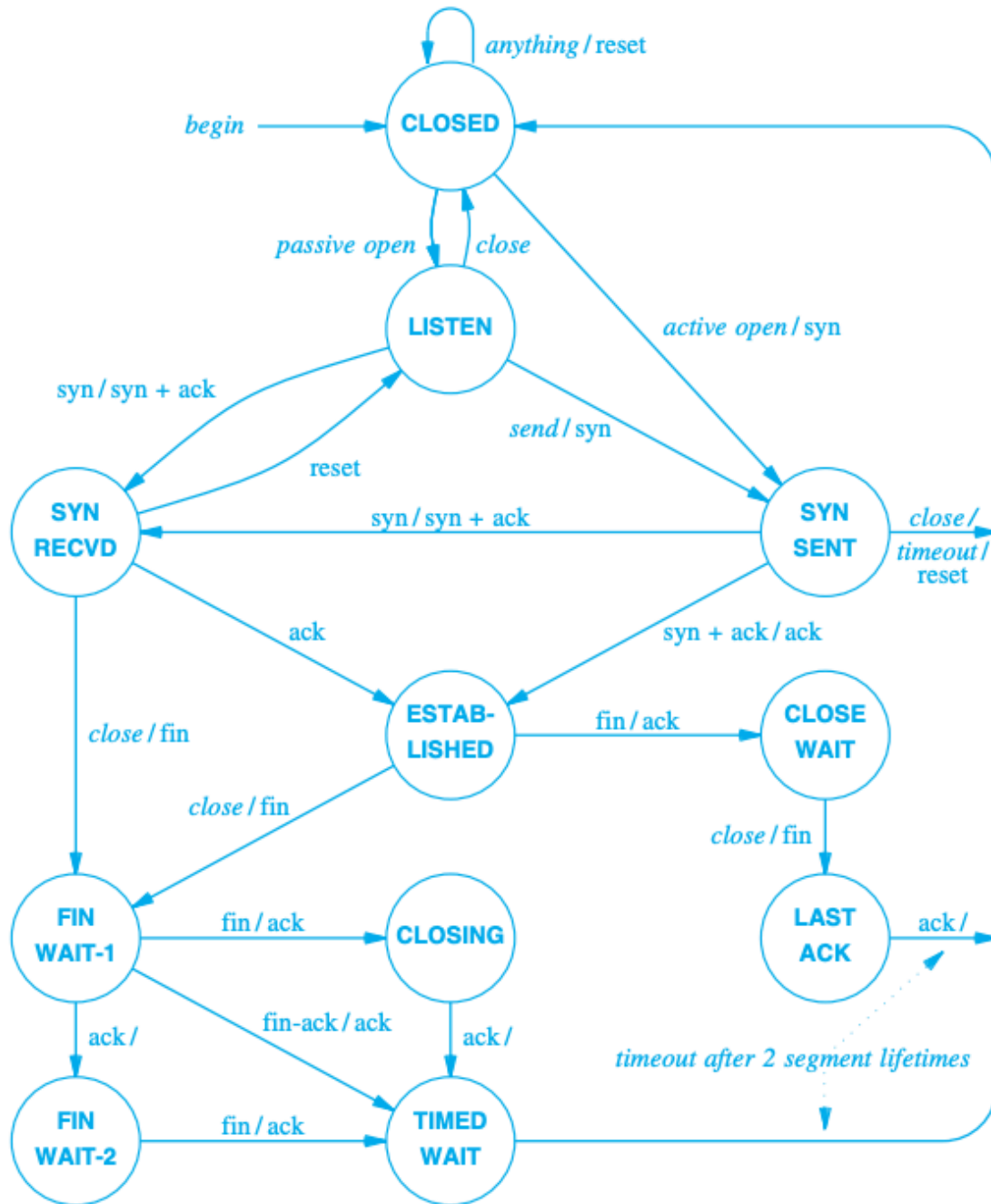


Figure 11.16 The finite state machine for TCP. Labels on state transitions show the input that caused the transition followed by the output if any.

En la figura, la etiqueta de cada transición de estado muestra qué la provoca y qué envía TCP al realizar esa transición. Por ejemplo, el software TCP en cada punto final comienza en el estado CLOSED. Los programas de aplicación deben emitir ya sea un comando de apertura pasiva (*passive open*) para esperar una conexión de otra máquina, o un comando de apertura activa (*active open*) para iniciar una conexión. Un comando de apertura activa fuerza una

transición del estado CLOSED al estado SYN SENT. Al seguir la transición hacia SYN SENT, TCP emite un segmento SYN. Cuando el otro lado devuelve un segmento con los bits SYN y ACK activados, TCP pasa al estado ESTABLISHED, emite un ACK, y comienza la transferencia de datos.

El estado TIMED WAIT revela cómo TCP maneja algunos de los problemas derivados de la entrega no confiable. TCP mantiene una noción de tiempo de vida máximo del segmento (MSL, maximum segment lifetime), el tiempo máximo que un segmento antiguo puede permanecer vivo en una internet. Para evitar que segmentos de una conexión anterior interfieran con una conexión actual, TCP pasa al estado TIMED WAIT después de cerrar una conexión. Permanece en ese estado durante el doble del tiempo de vida máximo del segmento antes de eliminar su registro de la conexión. Si llegan segmentos duplicados para la conexión durante el intervalo de espera, TCP los rechazará. Sin embargo, para manejar los casos en que se perdió la última confirmación, TCP confirma los segmentos válidos y reinicia el temporizador. Debido a que el temporizador permite que TCP distinga las conexiones antiguas de las nuevas, evita que TCP envíe un reinicio (reset) en respuesta a segmentos retrasados de una conexión antigua (por ejemplo, si el otro extremo retransmite un segmento FIN).

Forzar la entrega de datos

Dijimos que TCP tiene libertad para dividir el flujo de datos en segmentos para su transmisión, sin importar el tamaño de las transferencias que usan las aplicaciones. La principal ventaja de permitir que TCP elija una división es la eficiencia. TCP puede acumular suficientes octetos en un buffer para hacer segmentos razonablemente largos, reduciendo la alta sobrecarga que ocurre cuando los segmentos contienen solo unos pocos octetos de datos. Aunque el uso de buffers mejora el rendimiento de la red, puede interferir con algunas aplicaciones. Consideremos el uso de una conexión TCP para pasar caracteres desde una terminal interactiva a una máquina remota. El usuario espera una respuesta instantánea a cada pulsación de tecla. Si el TCP emisor almacena los datos en buffer, la respuesta puede retrasarse, quizás durante cientos de pulsaciones. De manera similar, debido a que el TCP receptor puede almacenar datos en buffer antes de ponerlos a disposición del programa de aplicación en su extremo, forzar al emisor a transmitir datos puede no ser suficiente para garantizar la entrega.

Para dar cabida a los usuarios interactivos, TCP proporciona una operación push que una aplicación puede usar para forzar la entrega de los octetos actualmente presentes en el flujo

sin esperar a que el buffer se llene. La operación push hace más que forzar al TCP local a enviar un segmento. También solicita a TCP que active el bit PSH en el campo de código del segmento, para que los datos se entreguen al programa de aplicación en el extremo receptor. Así, una aplicación interactiva usa la función push después de cada pulsación de tecla. De manera similar, las aplicaciones que controlan una pantalla remota pueden usar la función push para garantizar que los datos se envíen a través de la conexión de manera oportuna y se pasen de inmediato a la aplicación del otro lado.

Números de puerto TCP reservados

Al igual que UDP, TCP usa una combinación de números de puerto de protocolo asignados de manera estática y dinámica. Un conjunto de puertos bien conocidos (well-known ports) han sido asignados por una autoridad central para servicios de uso común (por ejemplo, servidores web y servidores de correo electrónico). Otros números de puerto están disponibles para que un sistema operativo los asigne a aplicaciones locales según sea necesario. Actualmente existen muchos puertos TCP bien conocidos. La figura 11.17 enumera algunos de los puertos TCP asignados actualmente.

Port	Keyword	Description
0		Reserved
7	echo	Echo
9	discard	Discard
13	daytime	Daytime
19	chargen	Character Generator
20	ftp-data	File Transfer Protocol (data)
21	ftp	File Transfer Protocol
22	ssh	Secure Shell
23	telnet	Terminal connection
25	smtp	Simple Mail Transport Protocol
37	time	Time
53	domain	Domain name server
80	www	World Wide Web
88	kerberos	Kerberos security service
110	pop3	Post Office Protocol vers. 3
123	ntp	Network Time Protocol
161	snmp	Simple Network Management Protocol
179	bgp	Border Gateway Protocol
443	https	Secure HTTP
860	iscsi	iSCSI (SCSI over IP)
993	imaps	Secure IMAP
995	pop3s	Secure POP3
30301	bittorrent	BitTorrent service

Figure 11.17 Examples of currently assigned TCP port numbers.

Debemos señalar que, aunque los números de puerto TCP y UDP son independientes, los diseñadores han elegido usar los mismos números de puerto enteros para cualquier servicio accesible tanto desde UDP como desde TCP. Por ejemplo, un servidor de nombres de dominio puede accederse tanto con TCP como con UDP. Al usar cualquiera de los dos protocolos, una aplicación cliente puede usar el mismo número de puerto, 53, porque la IANA asignó el 53 como el puerto del servicio de nombres de dominio tanto para TCP como para UDP.